URIB Canonical Architecture Master Specification

Universal Bridge Stack — Canonical Reference v1.0

Document Title	URIB Canonical Architecture Master Specification
Version	1.0 — AUTHORITATIVE / CANONICAL
Classification	Developer & Integrator Reference
Status	AUTHORITATIVE
Authored By	leon — Principal Architect, URIB Protocol
Location	Spicewood, TX, United States
Date Issued	Thursday, 09 July 2026 — 01:03 CDT
Platform Context	Base44 — Full Working Demo Active
Audience	Protocol Engineers, Integrators, Smart Contract Developers, Platform Architects

NORMATIVE AUTHORITY NOTICE

This document constitutes the single authoritative canonical specification for the Universal Identity

&

Resource Bridge (URIB) architecture. All prior drafts, partial specifications, and informal design notes are superseded by this document. Implementors and integrators MUST treat this document as the ground truth. The key words "MUST," "MUST NOT," "REQUIRED," "SHALL," "SHALL NOT," "SHOULD," "SHOULD

NOT," "RECOMMENDED," "MAY," and "OPTIONAL" in this document are to be interpreted as defined in RFC 2119.

Abstract

The Universal Identity & Resource Bridge (URIB) is a universal semantic identity and resource bridge architecture that provides deterministic, auditable, cross-rail settlement across heterogeneous systems, blockchains, and data rails. URIB defines a seven-layer Universal Bridge Stack through which any asset, identity, claim, or semantic object can be canonically represented, cryptographically anchored, and deterministically bridged across any transport or ledger substrate — including Bitcoin, Ethereum, XRPL, Solana, IPFS, and arbitrary HTTPS endpoints — without semantic loss, identity fragmentation, or settlement ambiguity. URIB achieves this through four foundational primitives: the Canonical Form (a rail-agnostic object representation), the Semantic Anchor (an ontological binding that persists across all rails), ThreadZero (an immutable root causal identity chain), and the Taproot Commitment Layer (a generalized cryptographic commitment tree that anchors all bridge proofs). This specification defines all schemas, algorithms, invariants, lifecycle rules, security properties, and integration interfaces required to build fully conformant URIB implementations.

Table of Contents

---

1. Executive Overview

> *This section provides a normative summary of the URIB system, its purpose, its design philosophy, and the invariants that all conformant implementations must uphold.*

1.1 What URIB Is

The Universal Identity & Resource Bridge (URIB) is a protocol-agnostic, semantically-anchored identity and resource bridge architecture. URIB does not impose a new ledger, a new token standard, or a new consensus mechanism. Instead, it defines a universal coordination layer — a bridge stack — that sits above any underlying transport or ledger substrate and provides a consistent, auditable, semantically-stable representation of any object that flows through it.

URIB operates on the principle that every real-world asset, digital identity, claim, event, or relationship can be expressed as a **Canonical Object** — a rail-agnostic, hash-stable, semantically-anchored data structure whose meaning is preserved regardless of which ledger or transport carries it at any given moment. Once an object has a Canonical Form, it can be bridged, settled, verified, and referenced across heterogeneous rails with full cryptographic guarantees and without semantic loss.

A fully deployed URIB implementation provides the following capabilities:

- Canonical representation of any asset, identity, or semantic object.
- Deterministic, lossless mapping of any canonical object between any two supported rails.
- Cryptographically provable bridge commitments anchoring cross-rail state.
- An immutable root identity thread (ThreadZero) from which all derived identities emanate.
- Globally-monotonic ordinal sequencing for all canonical events across all rails.
- Settlement equivalence proofs that allow finality on one rail to be accepted as evidence of finality on any other.

1.2 Core Value Proposition

Any asset, identity, or semantic object can be **canonically represented**, **deterministically bridged**, and **settled across any rail** — with a complete, auditable, non-repudiable proof chain that satisfies compliance, security, and operational requirements in any domain.

URIB eliminates the three core failure modes of today's cross-chain and cross-system integration landscape:

- **Semantic Drift:** Objects lose meaning as they cross system boundaries. URIB's Semantic Anchor prevents this by binding every object to an ontological concept that persists across all rails.
- **Identity Fragmentation:** Entities acquire multiple incompatible identities on different systems. ThreadZero prevents this by establishing a single immutable root identity with cryptographic continuity proofs.
- **Settlement Ambiguity:** Finality on one rail does not imply finality on another. The Taproot Commitment Layer and Bridge Commitment lifecycle resolve this by providing verifiable, cross-rail settlement equivalence proofs.

1.3 Audience

This specification is intended for the following audiences:

- **Protocol Engineers** who implement Rail Adapters, Ordinal Registries, and Commitment Validators.
- **Smart Contract Developers** who deploy on-chain components of URIB Bridge Commitments and Taproot Anchors.
- **Platform Architects** who design systems that consume or produce URIB Canonical Objects.
- **Integrators** who connect existing systems (DeFi protocols, enterprise databases, identity providers, IoT platforms) to the URIB bridge stack.
- **Security Auditors** who verify conformance, threat resilience, and invariant preservation.

1.4 Key Invariants Summary

The following invariants are non-negotiable. Any implementation that violates any of these invariants is non-conformant. Full formal statements appear in Section 5.

- **I-1 Canonical Uniqueness:** Every canonical object has exactly one C-Hash. No two distinct objects share a C-Hash.
- **I-2 Semantic Consistency:** An object's Semantic ID resolves to the same semantic concept on every rail, at all times.
- **I-3 Ordinal Monotonicity:** Ordinals are strictly increasing within any rail and causally ordered across rails.
- **I-4 Identity Continuity:** A ThreadZero identity persists through any rail transition without identity loss or duplication.
- **I-5 Non-Duplication:** No canonical object can be in an active state on more than one rail simultaneously.
- **I-6 Bridge Soundness:** A bridge commitment is valid if and only if both source and destination state hashes are canonical and the ordinal window is within the finality window.

- **I-7 Settlement Equivalence:** Finality on any rail R1 implies verifiable proof of equivalent settlement on any other rail R2, given a FINALIZED bridge commitment.
- **I-8 Semantic Losslessness:** Rail mapping preserves all semantic tags. No tag may be dropped or altered during the mapping function ϕ.

1.5 Architecture Philosophy

URIB is built on three foundational principles:

Canonical-First: No object enters the URIB bridge stack without first being assigned a canonical form. The canonical form is the ground truth. All rail-specific representations are derived views of the canonical form, never the other way around.

Semantic-Anchored: Every canonical object carries a Semantic ID that binds it to a concept in the URIB Semantic Graph. This ontological binding is immutable once assigned. It persists across all rails, all versions, and all time. Semantics are not inferred from rail context — they are declared at creation and cryptographically committed.

Settlement-Proven: No bridge operation is considered complete until a cryptographic settlement proof exists and has been anchored on both the source and destination rails. Optimistic assumptions are supported as a performance optimization but must be backed by a complete proof within the finality window.

2. Terminology & Definitions Glossary

> *This section provides the authoritative definition of all terms used throughout this specification. All terms defined here are to be understood in their URIB-specific sense unless explicitly noted otherwise.*

Term	Definition
URIB	Universal Identity & Resource Bridge. The complete seven-layer protocol architecture defined by this specification, providing canonical representation, semantic anchoring, cross-rail mapping, bridge commitment, and settlement equivalence for any asset, identity, or semantic object.
Canonical Form	The rail-agnostic, hash-stable, deterministically serialized representation of a semantic object. The canonical form is the single source of truth from which all rail-specific representations are derived. Two objects with identical canonical forms are considered identical regardless of where or how they were created.
Canonical Object	An object that has been assigned a canonical form, a Canonical ID (CID), a Semantic ID (SID), an ordinal, and at least one rail anchor. A canonical object is the atomic unit of the URIB bridge stack.
Canonical Hash (C-Hash)	The globally unique identifier computed from a canonical object's fields using the formula: `SHA3-256(canonical_id || ordinal || payload_hash || sorted(rail_anchors))`. The C-Hash is deterministic and collision-resistant. It serves as the primary key for all canonical objects in the URIB registry.
Semantic Anchor	The binding that links a canonical object to a specific node in the URIB Semantic Graph. A semantic anchor is composed of the Semantic ID and the set of semantic tags assigned at creation. The semantic anchor is immutable once committed.
Semantic Tag	A typed label from the URIB semantic taxonomy applied to a canonical object to declare its ontological category. Valid tags are: IDENTITY, ASSET, CLAIM, EVENT, RELATIONSHIP, POLICY, COMMITMENT. An object may carry multiple tags if it belongs to multiple categories.
Semantic ID (SID)	A globally unique, stable identifier for a semantic concept in the URIB Semantic Graph. SIDs are URN-formatted strings of the form `urn:urib:sid:{namespace}:{concept_hash}`. A SID resolves to a node in the Semantic Graph and determines the object's meaning independent of rail context.

Term	Definition
ThreadZero	The primordial identity thread; the root causal chain from which all derived identity threads emanate. ThreadZero is created by a Genesis Event and can never be destroyed, only extended by appending new Thread Events in causal order. ThreadZero is the authoritative source of identity continuity for any URIB identity entity.
Taproot	The cryptographic commitment tree root that anchors all bridge proofs in URIB. Inspired by Bitcoin's Taproot construction but generalized for multi-rail contexts. The Taproot Root (TR) is the Merkle root of a set of sorted Commitment Leaves, and it represents the authoritative commitment state for a batch of bridge operations. The TR is published to each participating rail.
Ordinal	A monotonically-increasing sequence number assigned to every canonical event, globally unique within a rail and cross-rail referenceable through the Ordinal Namespace triple {rail_id, epoch, sequence}. Ordinals provide a total causal ordering of all events in the URIB system.
Rail	Any transport, ledger, or execution layer over which URIB objects flow. Examples include Bitcoin (UTXO model), Ethereum (EVM state trie), XRPL (XRP Ledger), Solana (Sealevel runtime), IPFS (content-addressed storage), and HTTPS (REST endpoints). Each rail has its own finality model, address format, asset model, and transaction structure.
Rail Map / Rail Mapping	The deterministic function $\phi(O, R_{src}, R_{dst}) \rightarrow O'$ that maps a canonical object O from its representation on source rail R_src to an equivalent representation on destination rail R_dst, without semantic loss. Rail mappings must be deterministic, lossless, invertible, and composable.
Cross-Rail Invariant	A property that must hold true for a canonical object regardless of which rail it exists on and regardless of what operations have been performed on it. Cross-rail invariants are the formal correctness conditions for the URIB bridge stack. Eight cross-rail invariants are defined in Section 5.
Bridge Commitment	A cryptographically-provable pledge, anchored on both the source and destination rails, that a canonical object's state on one rail corresponds exactly (or equivalently) to its state on another. A bridge commitment progresses through a defined state machine: PENDING → ANCHORED → PROVEN → FINALIZED → SETTLED (or REVERTED).
Identity Continuity	The guarantee that a URIB identity — represented by its ThreadZero — remains the same logical entity across all rails and all time. Identity continuity is proven by a Merkle path from any thread event back to the ThreadZero genesis block. No rail transition, key rotation, or system migration breaks identity continuity.
Settlement Equivalence	The property by which finality on any rail is treated as equivalent to finality on any other rail, modulo rail-specific finality windows. Settlement equivalence is achieved when a FINALIZED bridge commitment exists linking two rails and the Taproot Root is anchored on both. It does not require simultaneous confirmation on all rails.

Term	Definition
Semantic Object	Any entity (asset, identity, claim, event, relationship, policy, or commitment) that has been semantically tagged and assigned a SID in the URIB Semantic Graph. Every canonical object corresponds to exactly one semantic object.
Semantic Graph	A directed acyclic graph (DAG) where nodes are semantic objects and edges are typed relationships between them. The Semantic Graph is the global ontology of all concepts known to the URIB system. It is append-only and content-addressed.
Rail Adapter	The module or smart contract that interfaces between a specific rail's native format and the URIB Rail Normalization Layer. Each rail requires exactly one conformant Rail Adapter implementation. Rail Adapters are responsible for reading raw rail data, producing normalized Rail Objects, and publishing bridge commitments and Taproot Roots to the rail.
Primitive	An atomic, indivisible URIB object that cannot be decomposed into smaller canonical objects without semantic loss. Primitives are the leaves of the Semantic Graph. Examples include a single UTXO, a single public key, or a single signed claim.
Composite Object	A canonical object composed of two or more primitives or other canonical objects, unified under a single CID and SID. Composite objects are non-atomic and their Semantic Graph node has at least two outgoing edges.
Bridge Proof	A verifiable data structure that demonstrates that a specific bridge commitment is valid, final, and settled on both participating rails. A bridge proof consists of: {bridge_commitment, taproot_root, rail_anchor_R1, rail_anchor_R2, ordinal_proof, finality_attestation}.
Commitment Root	Synonymous with Taproot Root (TR) in the context of a specific batch of bridge commitments. The Commitment Root is the Merkle root of all commitment leaves in a batch, computed as `MerkleRoot(sorted(commitment_leaves))`.
Rail Anchor	A reference to a specific location on a specific rail where a canonical object or commitment has been published. A rail anchor is a tuple {rail_id, block_height_or_slot, tx_hash, output_index}. An object may have multiple rail anchors if it exists on multiple rails.
Cross-Rail Reference (XRR)	A stable pointer from an object on one rail to the same canonical object on another rail. XRRs are constructed from the CID plus both rail anchors and are verified via the bridge commitment linking them.
Merge Proof	A proof that two canonical objects have been combined into a single composite canonical object, with both constituent objects' provenance preserved. Merge Proofs are required when composite objects are formed from objects originating on different rails.
Fork Proof	A proof that a canonical object has been legitimately split into two or more descendant canonical objects, each with full provenance tracing back to the parent. Fork Proofs prevent unauthorized duplication by requiring that the parent object's active state be nullified before descendants are activated.

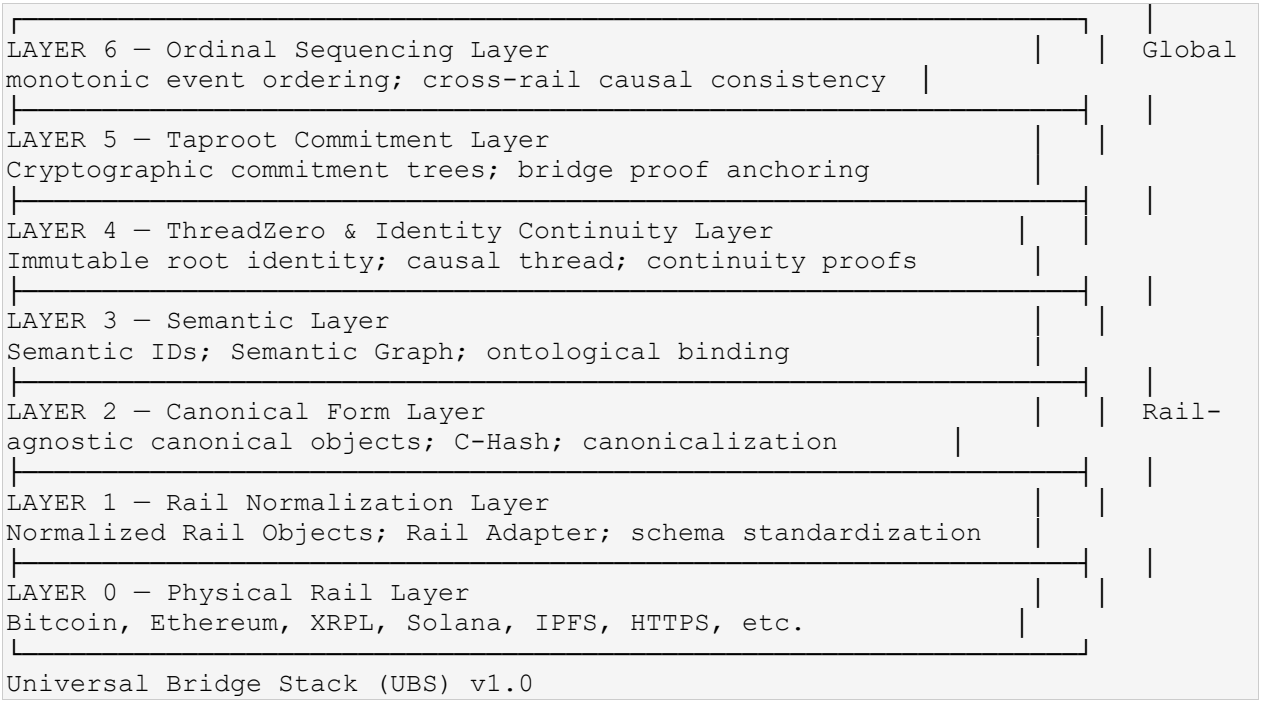
Term	Definition
Nullifier	A cryptographic value derived from a canonical object's CID and ordinal that, once published, marks the object as spent, consumed, or transferred. Publishing a nullifier prevents the same object from being used again on any rail (preventing double-spend). Nullifiers are irreversible.
Witness	A cryptographic attestation (signature or zero-knowledge proof) provided by an authorized party that validates a thread event, bridge commitment, or state transition. Multiple witnesses may be required for high-value operations (multi-sig threshold).
Epoch	A time interval used to partition ordinal sequences and anchor cross-rail causal ordering. Epoch boundaries are determined by the slowest-finality rail in any given bridge operation (typically Bitcoin). Epoch numbers are globally monotonic across all URIB operations.
Finality Window	The rail-specific time interval after which a transaction is considered irreversible. URIB bridge commitments must remain in the PROVEN state until the finality window of both participating rails has elapsed before transitioning to FINALIZED. See Section 8 for per-rail finality windows.
State Transition	A change in the status of a canonical object or bridge commitment, triggered by an authorized event and validated by one or more witnesses. State transitions are recorded as Thread Events and assigned ordinals.
Settlement Layer	The logical abstraction above all individual rails where settlement equivalence is established. The Settlement Layer does not correspond to any single rail; it is an emergent property of the URIB bridge stack's commitment and proof infrastructure.

---

3. Universal Bridge Stack Architecture

> *This section is the central architectural definition of URIB. It specifies all seven layers of the Universal Bridge Stack, their responsibilities, schemas, algorithms, and interactions. All conformant implementations must implement all seven layers.*

The Universal Bridge Stack (UBS) is a seven-layer architecture (Layer 0 through Layer 6) analogous in structure to the OSI network model. Each layer provides services to the layer above it and consumes services from the layer below it. Layers are decoupled by well-defined interfaces. No layer may access the internals of a non-adjacent layer directly.



Universal Bridge Stack (UBS) v1.0

3.1 Layer 0 — Physical Rail Layer

> *Layer 0 is the actual transport and ledger substrate. URIB does not control Layer 0; it interfaces with it through Rail Adapters.*

The Physical Rail Layer encompasses all underlying transport mechanisms and ledger substrates that URIB bridges across. Each rail has its own native data model, finality mechanism, address format, transaction structure, and fee model. URIB treats all rails as opaque at this layer — their native properties are relevant only to the Rail Adapter (Layer 1) that interfaces with them.

Layer 0 Properties:

- **Heterogeneous:** Rails differ radically in data model, consensus mechanism, and finality guarantees.
- **Non-standardized:** No two rails share a common data format or transaction schema at the native level.
- **Rail-specific finality:** Each rail has its own probabilistic or deterministic finality model.
- **Opaque to higher layers:** Layers 2 and above interact only with normalized Rail Objects, never raw rail data.

URIB Interface: A conformant Rail Adapter module per rail, implementing the interface defined in Section 4.3. The Rail Adapter is the only component permitted to interact directly with Layer 0.

3.2 Layer 1 — Rail Normalization Layer

> *Layer 1 converts raw rail data into normalized Rail Objects with a standardized schema, providing a uniform interface to the Canonical Form Layer above.*

The Rail Normalization Layer is responsible for translating the rail-specific native format of any transaction or state element into a **Rail Object** — a standardized, schema-validated data structure that all higher layers can process without rail-specific logic.

3.2.1 Rail Object Schema

```
// Rail Object — Normalized representation of a single rail event {
"schema_version": "1.0",   "rail_id":        string,          // Globally
unique rail identifier (e.g., "btc-mainnet", "eth-mainnet")   "block_height":
integer,         // Block height or slot number at time of event   "tx_hash":
hex_string,      // Native transaction hash (rail-specific format, hex-
encoded)   "output_index":   integer | null,  // Output index within
transaction (null for account-model rails)   "payload":        object,
// Rail-native structured payload (UTXO, EVM log, XRPL object, etc.)
"payload_bytes":  hex_string,      // Canonical hex encoding of the payload
bytes   "timestamp_utc":  ISO8601_string,  // UTC timestamp of block/slot
containing this event   "finality_status": enum {
"UNCONFIRMED",                        "CONFIRMING",
"FINALIZED"                     },   "confirmations":  integer,         //
Number of confirmations / epochs since inclusion   "rail_address":   string,
// Originating address in rail-native format   "adapter_version": string
// Version of the Rail Adapter that produced this object }
```

3.2.2 Rail Adapter Responsibilities

- Read raw events from the underlying rail (blocks, transactions, logs, ledger entries).
- Validate that raw events conform to rail-specific schema expectations.
- Produce a conformant Rail Object for each relevant event.
- Track confirmation depth and update `finality_status` as confirmations accumulate.
- Publish Taproot Roots and bridge commitments back to the rail (write direction).
- Expose the standard Rail Adapter interface defined in Section 4.3.

IMPLEMENTATION NOTE

Rail Adapters MUST be stateless with respect to semantic meaning. A Rail Adapter reads and writes rail data; it does not interpret semantic content. Semantic interpretation is the exclusive responsibility of Layer 3.

3.3 Layer 2 — Canonical Form Layer

> *Layer 2 transforms normalized Rail Objects into rail-agnostic Canonical Objects and assigns each a globally unique, deterministically-computed C-Hash.*

3.3.1 Canonical Object Schema

```
// Canonical Object — The atomic unit of the URIB bridge stack {
"schema_version":  "1.0",   "canonical_id":    string,           // UUID v5
(SHA-1 namespace + SID + ordinal)   "semantic_id":     string,           //
SID — urn:urib:sid:{namespace}:{concept_hash}   "ordinal":         {
// Ordinal Namespace triple                          "rail_id":   string,
"epoch":     integer,                          "sequence":  integer
},   "payload_hash":    hex_string,       // SHA3-256 of canonical payload
bytes (UTF-8, sorted fields)   "rail_anchors":    [                 // Array
of rail anchors (at least one required)                          {
"rail_id":      string,                           "block_height": integer,
"tx_hash":      hex_string,                           "output_index": integer
| null                          }                       ],   "created_at":
ISO8601_string,   // UTC creation timestamp   "version":         integer,
// Monotonically-increasing version counter (starts at 1)   "status":
enum {                         "ACTIVE",                          "BRIDGING",
"NULLIFIED",                         "TOMBSTONED"                          },
"c_hash":          hex_string,       // Canonical Hash — computed, not stored
in signed payload   "metadata": {     "created_by":    string,           //
Thread ID of creating identity     "tags":          [string],         //
Semantic tags (IDENTITY, ASSET, CLAIM, EVENT, ...)     "description":
string | null,    // Human-readable description     "custom":        object |
null     // Arbitrary extension fields (must be hash-stable)   } }
```

3.3.2 Canonicalization Algorithm

The canonicalization algorithm converts a Rail Object into a Canonical Object payload suitable for C-Hash computation. The algorithm is mandatory and must be followed exactly to ensure deterministic output.

1. **Field Selection:** Extract all semantically meaningful fields from the Rail Object. Exclude ephemeral fields (`confirmations`, `finality_status`, `adapter_version`).
2. **UTF-8 Normalization:** All string values MUST be normalized to Unicode NFC form and encoded as UTF-8.
3. **Deterministic Field Ordering:** All JSON object keys MUST be sorted lexicographically (byte-level, ascending). Arrays are ordered by content hash (ascending). No whitespace between tokens.
4. **Numeric Normalization:** All integers serialized as decimal strings without leading zeros. Floats are not permitted in canonical payloads; use integer representations with explicit precision metadata.
5. **Hash Computation:** Apply SHA3-256 to the UTF-8 bytes of the deterministically serialized JSON to produce `payload_hash`.
6. **C-Hash Computation:** See Section 3.3.3.

3.3.3 C-Hash Computation

```
// C-Hash Computation Formula // All inputs are UTF-8 byte strings unless
otherwise noted. // Concatenation uses the || operator. //
sorted(rail_anchors) = lexicographic sort of each anchor's tx_hash hex string
c_hash = SHA3-256(     canonical_id_bytes     ||   // UTF-8 bytes of
canonical_id string     ordinal_bytes          ||   // Big-endian 8-byte
epoch + 8-byte sequence     payload_hash_bytes     ||   // 32 raw bytes of
SHA3-256 payload hash     sorted_anchors_hash         // SHA3-256 of
concatenated sorted anchor tx_hashes )
```

3.3.4 Worked Example — Ethereum Asset Canonicalization

The following example shows a raw Ethereum ERC-20 transfer being canonicalized into a URIB Canonical Object.

Step 1: Raw Ethereum Event (Layer 0)

```
// Raw EVM Transfer Event (ERC-20) {   "network":       "mainnet",
"block_number":  20048221,   "tx_hash":       "0xabc123...def456",
"log_index":     3,   "contract":
"0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48",  // USDC   "from":
"0xDEAD...BEEF",   "to":            "0xCAFE...BABE",   "value":
"1000000",   // 1.000000 USDC (6 decimals)   "block_time":    "2026-07-
09T01:03:00Z" }
```

Step 2: Rail Normalization (Layer 1) — Rail Object

```
{   "schema_version":  "1.0",   "rail_id":         "eth-mainnet",
"block_height":    20048221,   "tx_hash":         "0xabc123def456...",
"output_index":    null,   "payload":         { "contract":"0xA0b8...",
"from":"0xDEAD...", "to":"0xCAFE...", "value":"1000000" },   "payload_bytes":
"7b22636f6e7472616374223a...",   "timestamp_utc":   "2026-07-09T01:03:00Z",
"finality_status": "FINALIZED",   "confirmations":   32,   "rail_address":
"0xDEAD...BEEF",   "adapter_version": "1.0.0" }
```

Step 3: Canonical Form (Layer 2) — Canonical Object

```
{   "schema_version":  "1.0",   "canonical_id":    "c7f3a91b-0042-5e8d-b3c2-
1a9f4e6d8b02",   "semantic_id":     "urn:urib:sid:asset:sha3256-usdc-
transfer-v1",   "ordinal": {     "rail_id":   "eth-mainnet",     "epoch":
4412,     "sequence":  8820391   },   "payload_hash":
"e3b0c44298fc1c149afbf4c8996fb924...",   "rail_anchors": [{     "rail_id":
"eth-mainnet",     "block_height": 20048221,     "tx_hash":
"0xabc123def456...",     "output_index": null   }],   "created_at":   "2026-
07-09T01:03:00Z",   "version":      1,   "status":       "ACTIVE",
"c_hash":       "9f86d081884c7d659a2feaa0c55ad015...",   "metadata": {
"created_by": "thread:tz:0001-genesis",     "tags":       ["ASSET", "EVENT"],
"description":"USDC transfer 1.000000 on Ethereum mainnet"   } }
```

3.4 Layer 3 — Semantic Layer

> *Layer 3 binds every canonical object to a node in the URIB Semantic Graph, assigning an immutable Semantic ID that persists across all rails and all time.*

3.4.1 Semantic Anchoring

Every canonical object MUST carry a Semantic ID (SID) before it is considered fully canonicalized. The SID is assigned by the Semantic Layer upon first canonicalization and is never reassigned. The SID format is:

```
SID Format: urn:urib:sid:{namespace}:{concept_hash}  Where:   namespace    =
lowercase alphanumeric identifier (e.g., "asset", "identity", "claim")
concept_hash = SHA3-256(canonical_concept_definition_bytes) truncated to 32
hex chars  Example:   urn:urib:sid:identity:a3f7c291e8b04d12
urn:urib:sid:asset:9e1b3c7f2d4a0e58   urn:urib:sid:claim:7c4d2a1f9b3e6d08
```

3.4.2 Semantic Tag Taxonomy

Tag	Description	Valid Relationships	Example
IDENTITY	A logical entity (person, organization, device, service) with an associated ThreadZero.	OWNS, CONTROLS, IS_DELEGATED_BY, REVOKES	A DID-anchored user identity
ASSET	A transferable, quantifiable resource with a defined value model and ownership state.	OWNED_BY, TRANSFERRED_TO, LOCKED_IN, DERIVED_FROM	A Bitcoin UTXO, ERC-20 token balance
CLAIM	A verifiable assertion made by one identity about another entity.	ASSERTS, REFUTES, VERIFIED_BY, EXPIRES_ON	A KYC attestation, a verifiable credential
EVENT	A state change or occurrence at a specific ordinal in time.	CAUSED_BY, RESULTS_IN, PRECEDES, FOLLOWS	A token transfer, a bridge operation trigger
RELATIONSHIP	A typed, directed association between two or more semantic objects.	LINKS, MEDIATES, GOVERNS	Ownership link between identity and asset

Tag	Description	Valid Relationships	Example
POLICY	A set of rules or constraints governing how other semantic objects may behave or be used.	GOVERNS, ENFORCES, PERMITS, DENIES	A transfer policy, a compliance rule
COMMITMENT	A bridge commitment or promise of future state, with cryptographic proof obligations.	COMMITS_TO, PROVEN_BY, SETTLED_BY, REVERTS	A cross-rail bridge commitment

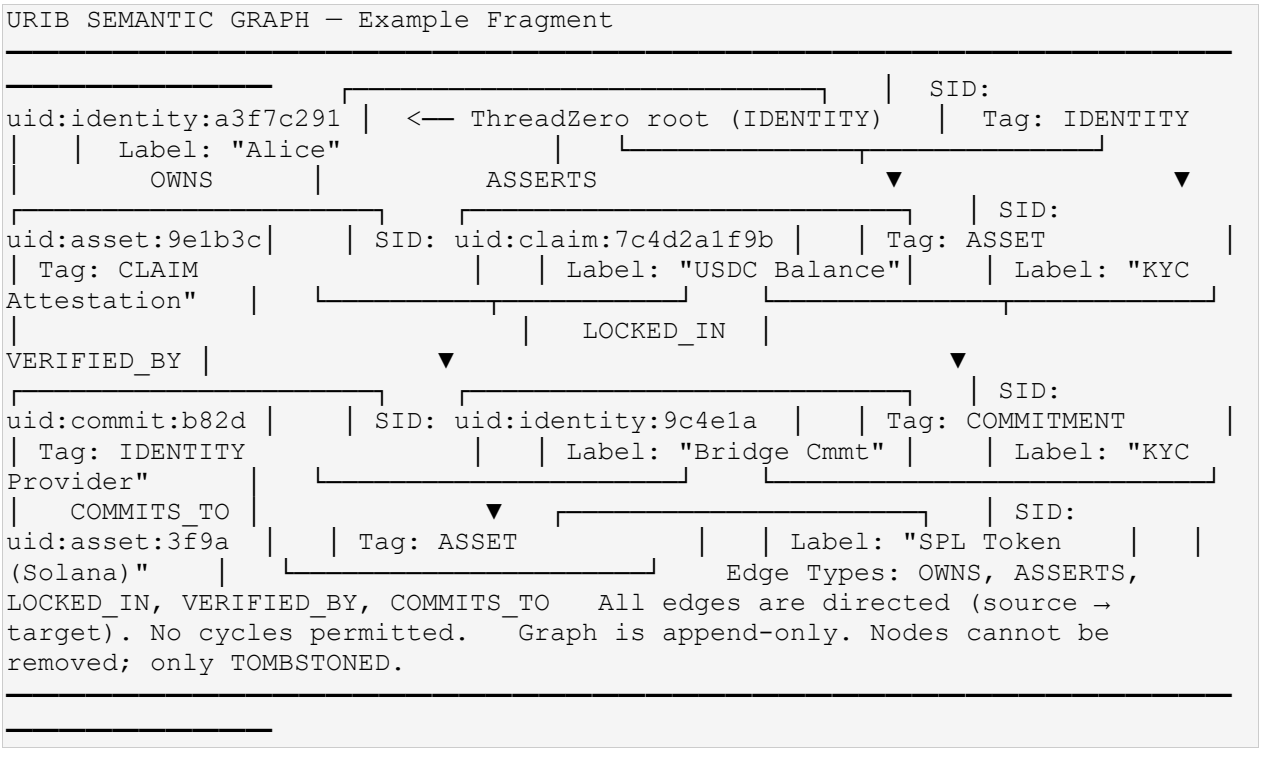
3.4.3 Semantic Equivalence

Two canonical objects O1 and O2 are **semantically equivalent** if and only if:

7. `SID(O1) == SID(O2)`, AND
8. Both SIDs resolve to the same root node in the Semantic Graph, AND
9. Neither object has been nullified or tombstoned.

Semantic equivalence does not imply state equivalence. Two objects may be semantically equivalent while having different `payload_hash` values (representing different states of the same semantic concept).

3.4.4 Semantic Graph — ASCII Diagram



3.5 Layer 4 — ThreadZero & Identity Continuity Layer

> *Layer 4 establishes and maintains the immutable root identity thread (ThreadZero) for every URIB identity entity and provides cryptographic continuity proofs for all identity operations.*

3.5.1 ThreadZero Deep-Dive

ThreadZero is the primordial identity thread. It is created exactly once per identity entity, during the Genesis Event. Once created, ThreadZero has the following invariant properties:

- **Immutable Origin:** The genesis block of a ThreadZero is cryptographically committed and can never be altered or revoked. It is the absolute anchor for all identity proofs.
- **Causal Append-Only:** New Thread Events are appended to the end of ThreadZero by including the hash of the previous event (`parent_hash`). Events can never be inserted, reordered, or deleted.
- **Cross-Rail Synchronized:** Thread Events are assigned ordinals and rail anchors. When an identity operates across multiple rails, all corresponding thread events reference the same ThreadZero ID and maintain cross-rail causal ordering via the Ordinal layer.
- **Indestructible:** A ThreadZero cannot be destroyed. It can only be TOMBSTONED, which adds a final event indicating deactivation. Even a tombstoned ThreadZero retains all its history and can be inspected for audit purposes.

3.5.2 Thread Event Schema

```
// Thread Event — An atomic event on a ThreadZero or derived thread {
"schema_version":   "1.0",   "thread_id":        string,         //
ThreadZero ID (urn:urib:thread:{genesis_hash})   "event_ordinal": {
"rail_id":        string,     "epoch":          integer,     "sequence":
integer   },   "event_type":       enum {                          "GENESIS",
// First event; creates ThreadZero                          "KEY_ROTATION",
// Public key change                          "RAIL_BRIDGE",     // Identity
crosses to a new rail                          "CLAIM_ASSERT",    // Identity
makes a claim                          "ASSET_BIND",      // Asset bound to
this identity                          "DELEGATION",      // Derived thread
created                          "REVOCATION",      // Delegation or claim
revoked                          "TOMBSTONE"        // Identity deactivated
},   "parent_hash":      hex_string,     // SHA3-256 of previous event's
serialized bytes   "payload_hash":     hex_string,     // SHA3-256 of event-
specific payload   "rail_anchor": {     "rail_id":        string,
"block_height":   integer,     "tx_hash":        hex_string,
"output_index":   integer | null   },   "witness_sigs":     [
// One or more Ed25519 signatures                          {
"signer_id":  string,    // ThreadZero ID of signer
"public_key": hex_string, // Ed25519 public key
"signature":  hex_string  // Ed25519 signature over event bytes
}                       ],   "derived_from":     string | null   // Parent
thread ID (null for ThreadZero genesis) }
```

3.5.3 ThreadZero Genesis Event

The Genesis Event is the first Thread Event on a new ThreadZero. It is constructed as follows:

10. Generate a new Ed25519 key pair `(sk, pk)` for the identity.
11. Construct the genesis payload: `{ "public_key": pk_hex, "created_at": utc_timestamp, "namespace": namespace, "metadata": {...} }`.
12. Compute `payload_hash = SHA3-256(canonical_serialize(genesis_payload))`.
13. Set `parent_hash = 0x00...00` (32 zero bytes — the null predecessor).
14. Set `event_type = "GENESIS"`.
15. Assign an ordinal from the Ordinal Layer (Layer 6).
16. Sign the event bytes with `sk` to produce `witness_sig`.
17. Compute the **ThreadZero ID**: `urn:urib:thread:{ SHA3-256(genesis_event_bytes).hex() }`.
18. Publish the genesis event to at least one rail, obtaining a `rail_anchor`.
19. Register the ThreadZero ID and its genesis anchor in the URIB Identity Registry.

3.5.4 Identity Continuity Proof

A continuity proof for a thread event E demonstrates that E belongs to a specific ThreadZero. It is a Merkle path from E back to the genesis event:

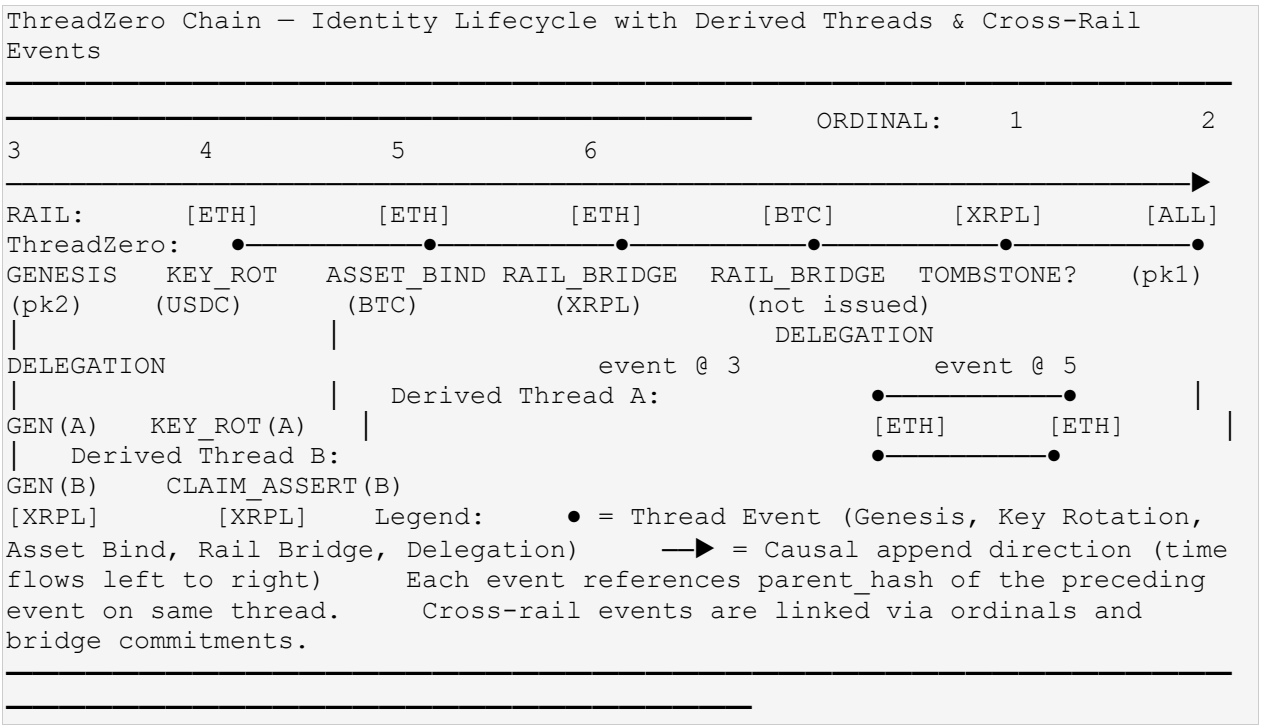
```
ContinuityProof = {   "thread_id":    string,        // The ThreadZero ID
"event_hash":   hex_string,    // Hash of the event being proven
"genesis_hash": hex_string,    // Hash of the genesis event   "merkle_path":
[              // Ordered list of sibling hashes from E to genesis
{                     "hash":     hex_string,
"position": "LEFT" | "RIGHT"                       }                   ],
"merkle_root":  hex_string     // Computed Merkle root (must equal thread
state root) }
```

3.5.5 Derived Threads

An identity may create **Derived Threads** — sub-identity threads that branch from ThreadZero or from another derived thread. Derived threads inherit the identity continuity guarantees of their parent but operate with independent key pairs and ordinal sequences. Rules:

- A derived thread MUST reference its parent thread ID in `derived_from`.
- The parent thread MUST publish a DELEGATION event at the same ordinal as the derived thread's GENESIS event.
- Derived threads MUST NOT merge with each other or with ThreadZero (the fork prohibition).
- A derived thread can be revoked by the parent thread via a REVOCATION event.

3.5.6 ThreadZero Chain Diagram



3.6 Layer 5 — Taproot Commitment Layer

> *Layer 5 provides a generalized cryptographic commitment tree (the Taproot) that anchors all bridge proofs, batches multiple commitments efficiently, and enables selective revelation of commitment details without exposing the full commitment batch.*

3.6.1 Taproot Architecture

The URIB Taproot is a binary Merkle tree where:

- **Leaves** are individual Bridge Commitment leaf hashes.
- **Internal Nodes** are SHA3-256 hashes of their two child node hashes.
- **Root** is the Taproot Root (TR) — the single 32-byte hash that represents the entire commitment batch.

The tree is constructed over lexicographically-sorted commitment leaves to ensure deterministic tree structure regardless of insertion order. This property is critical for cross-party verification: any two parties computing a Taproot Root over the same set of commitments must arrive at an identical TR.

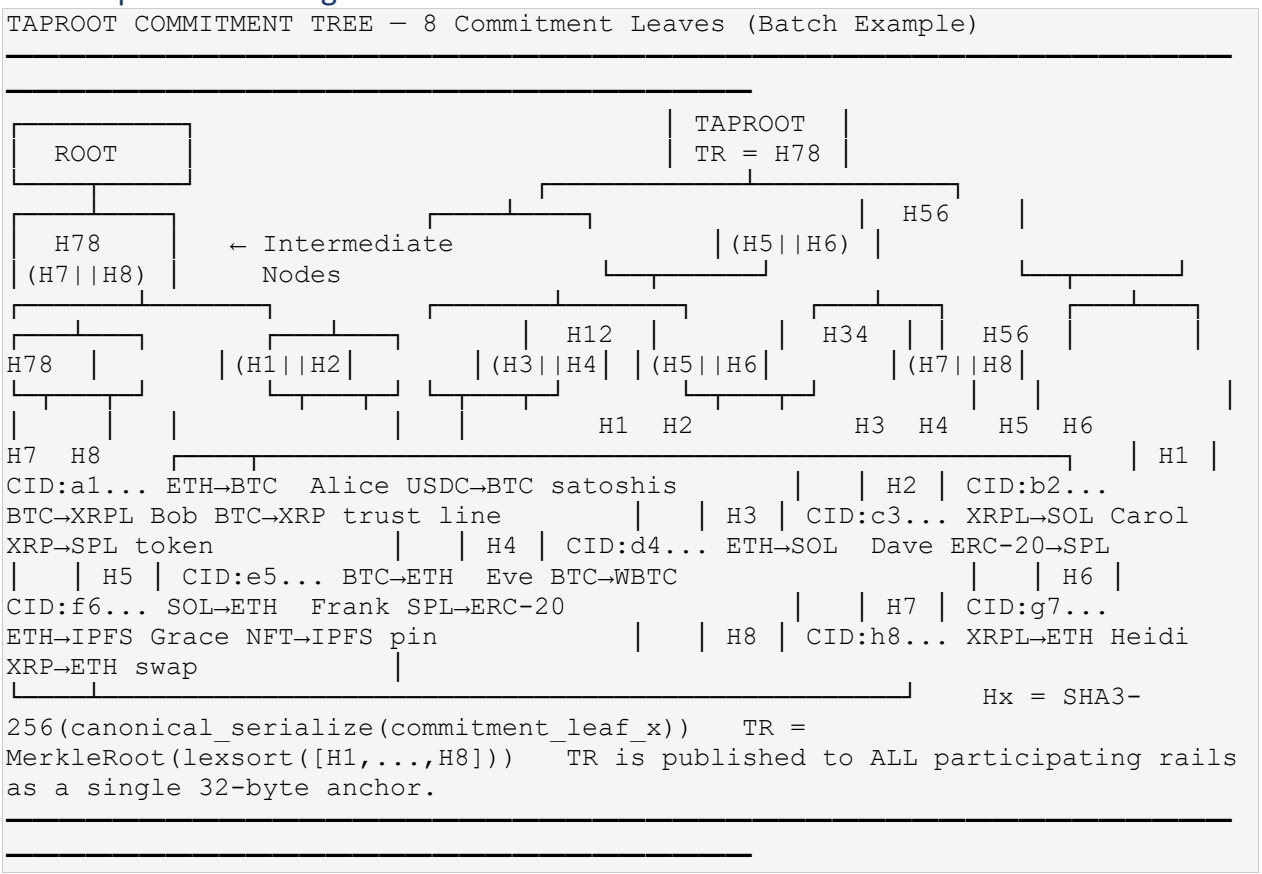
3.6.2 Commitment Leaf Schema

```
// Commitment Leaf — One entry in the Taproot tree {   "leaf_id":
string,        // UUID v5 (sha1_ns + canonical_id + ordinal_from)
"canonical_id":     string,        // CID of the canonical object being
bridged   "rail_from":        string,        // Source rail identifier
"rail_to":          string,        // Destination rail identifier
"state_hash_from":  hex_string,    // SHA3-256 of the object's canonical
state on rail_from   "state_hash_to":    hex_string,    // SHA3-256 of the
object's canonical state on rail_to   "ordinal_from": {     "rail_id":
string,     "epoch":          integer,     "sequence":       integer   },
"ordinal_to": {     "rail_id":        string,     "epoch":          integer,
"sequence":       integer   },   "timestamp_initiated": ISO8601_string,
"timestamp_finalized": ISO8601_string | null,   "leaf_hash":
hex_string     // SHA3-256 of canonical_serialize(this leaf, excluding
leaf_hash) }
```

3.6.3 Taproot Root Computation

```
// Taproot Root Computation Algorithm  function
compute_taproot_root(commitment_leaves):     // Step 1: Compute leaf hash for
each commitment leaf     leaf_hashes = [SHA3-256(canonical_serialize(leaf))
for leaf in commitment_leaves]      // Step 2: Sort leaf hashes
lexicographically (ascending byte order)     sorted_hashes =
lexicographic_sort(leaf_hashes)      // Step 3: Build Merkle tree bottom-up
//         If odd number of leaves, duplicate the last leaf hash.
current_level = sorted_hashes     while len(current_level) > 1:
next_level = []         for i in range(0, len(current_level), 2):
left  = current_level[i]             right = current_level[i+1] if i+1 <
len(current_level) else current_level[i]             node_hash = SHA3-
256(left || right)   // Raw byte concatenation
next_level.append(node_hash)         current_level = next_level      // Step
4: The single remaining hash is the Taproot Root (TR)     TR =
current_level[0]     return TR
```

3.6.4 Taproot Tree Diagram



3.6.5 Key-Path vs Script-Path Analogy

In Bitcoin Taproot, the key-path spend allows a transaction to be authorized by a single aggregated key (the simple case), while the script-path spend reveals a specific script leaf from the Taproot tree (the complex case). URIB adopts this duality:

- **Key-Path equivalent (Simple Settlement):** A bridge commitment is settled by a simple threshold-signature attestation without revealing the full Taproot tree. Suitable for routine, low-risk operations where all parties agree.
- **Script-Path equivalent (Provable Settlement):** A bridge commitment is settled by revealing the specific Commitment Leaf and its Merkle proof against the published Taproot Root. Required for disputed settlements, compliance audits, and high-value operations. Any verifier can independently confirm the leaf's membership in the TR.

3.6.6 Taproot Root Publication

After computing a Taproot Root for a commitment batch, the TR MUST be published to every rail that has at least one participating commitment leaf. Publication mechanisms are rail-specific:

Rail	TR Publication Mechanism
Bitcoin	OP_RETURN output in a standard P2TR transaction; 32-byte TR in script data
Ethereum	Event log emission from URIB Commitment Registry smart contract
XRPL	Memo field in an AccountSet transaction; TR hex-encoded
Solana	Program data account write via URIB Solana program (PDA)
IPFS	Content-addressed JSON document; CID published to participating rails
HTTPS	Signed JSON endpoint response; TR included in signed payload

3.7 Layer 6 — Ordinal Sequencing Layer

> *Layer 6 provides globally-monotonic ordinal sequence numbers for all canonical events, enabling total causal ordering of all URIB operations across all rails.*

3.7.1 Ordinal Namespace

Every canonical event is assigned an Ordinal, which is a globally unique triple:

```
OrdinalNamespace = {   "rail_id":  string,    // Identifies the rail on which this event was anchored   "epoch":    uint64,    // Epoch number (monotonically increasing; shared across rails)   "sequence": uint64     // Sequence number within the epoch on this rail (monotonic) }  Global uniqueness guaranteed by: (rail_id, epoch, sequence) triple is unique across all URIB events. Cross-rail ordering: events with lower epoch numbers always precede events with higher epoch numbers, regardless of rail. Within the same epoch, cross-rail ordering is determined by vector clock.
```

3.7.2 Cross-Rail Ordinal Ordering

Within a single epoch, events on different rails are ordered using a hybrid approach:

20. **Epoch Anchor:** At each epoch boundary, a synchronization event is published to all participating rails. The epoch boundary ordinal provides a global "checkpoint" that all rails agree on.
21. **Vector Clock:** Between epoch boundaries, cross-rail event ordering is maintained by a vector clock `V = {rail_id: sequence}` updated by each participating rail. A URIB event E1 causally precedes E2 if `V(E1)[r] ≤ V(E2)[r]` for all rails r.
22. **Conflict Resolution:** If two events have equal vector clock values (concurrent events), they are ordered by the hash of their respective thread IDs (deterministic tiebreaker).

3.7.3 Ordinal Assignment Table

The following table illustrates ordinal assignment across three rails over five epochs:

Epoch	Rail	Sequence	Event Type	Canonical ID (Short)	Cross-Rail Ref
4410	btc-mainnet	1	GENESIS	c7f3a91b	—
4410	eth-mainnet	1	ASSET_BIND	9e1b3c7f	→ btc:4410:1
4410	xrpl-mainnet	1	EPOCH_ANCHOR	—	btc:4410:1 + eth:4410:1
4411	eth-mainnet	1	BRIDGE_INIT	3f9a2b1c	—
4411	btc-mainnet	2	BRIDGE_RECV	3f9a2b1c	→ eth:4411:1
4411	xrpl-mainnet	2	CLAIM_ASSERT	7c4d2a1f	—
4411	xrpl-mainnet	3	EPOCH_ANCHOR	—	eth:4411:1 + btc:4411:2
4412	btc-mainnet	3	FINALIZE	3f9a2b1c	→ btc:4411:2 + eth:4411:1
4412	eth-mainnet	2	SETTLEMENT	3f9a2b1c	→ btc:4412:3
4412	xrpl-mainnet	4	EPOCH_ANCHOR	—	btc:4412:3 + eth:4412:2

3.7.4 Ordinal Proof

An Ordinal Proof demonstrates that a given ordinal is valid, non-duplicated, and causally consistent. It may be implemented as a Merkle inclusion proof against the Ordinal Registry state root, or as a compact ZK-SNARK proof (for privacy-preserving ordinal verification).

```
OrdinalProof = {   "ordinal":       OrdinalNamespace,   // The ordinal being
proven   "registry_root": hex_string,         // Ordinal Registry state root
at time of proof   "merkle_path":   [{ "hash": hex, "position":
"LEFT"|"RIGHT" }],   "witness_sig":   hex_string,         // Signature of an
authorized Ordinal Registry validator   "proof_type":    "MERKLE" | "ZK"
// Proof mechanism used }
```

4. Rail Mapping

> *This section defines the formal Rail Mapping function, its required properties, the Rail Map Registry, the Rail Adapter interface, and worked examples of cross-rail mappings.*

4.1 Formal Definition

The Rail Mapping function is formally defined as:

```
φ: (O, R_src, R_dst) → O'  Where:   O      = Canonical Object on source rail
R_src   R_src  = Source rail identifier   R_dst  = Destination rail
identifier   O'     = Canonical Object on destination rail R_dst  Required
Properties of φ:   1. DETERMINISTIC:  φ(O, R_src, R_dst) always produces the
same O' for the same inputs.                        Two independent calls with
identical inputs must produce bit-identical output.    2. LOSSLESS:
SID(O') == SID(O) and semantic_tags(O') ⊇ semantic_tags(O).
No semantic information present in O is absent in O'.    3. INVERTIBLE:
There exists φ⁻¹ such that φ⁻¹(O', R_dst, R_src) = O.                      The
inverse mapping must also be deterministic and lossless.    4. COMPOSABLE:
φ(φ(O, R1, R2), R2, R3) = φ(O, R1, R3)                     Chained mappings
produce the same result as a direct mapping.                      (Modulo
rail-specific precision constraints that are explicitly declared.)
```

4.2 Rail Map Registry

The Rail Map Registry is a canonical, content-addressed registry of all supported rail pairs and their mapping functions. Each entry specifies:

Field	Type	Description
`map_id`	string (UUID v5)	Unique identifier for this mapping entry
`rail_src`	string	Source rail identifier
`rail_dst`	string	Destination rail identifier
`asset_type_src`	string	Native asset type on source rail (e.g., UTXO, ERC20, TRUSTLINE)
`asset_type_dst`	string	Native asset type on destination rail
`mapping_fn_hash`	hex	SHA3-256 of the canonical mapping function implementation
`precision_loss`	boolean	True if precision loss is possible (must be declared)
`precision_notes`	string	Explanation of any precision constraints
`adapter_src`	string	Rail Adapter version required for source rail
`adapter_dst`	string	Rail Adapter version required for destination rail
`status`	enum	ACTIVE, DEPRECATED, EXPERIMENTAL

4.3 Rail Adapter Interface

All Rail Adapter implementations MUST expose the following interface. The pseudocode below is implementation-language-agnostic; implementations in Rust, Go, TypeScript, or Solidity must map each method to the appropriate language construct.

```
// Rail Adapter Interface — v1.0 // All methods are synchronous unless marked async.  interface RailAdapter {    // IDENTIFICATION   rail_id():
string              // Returns the canonical rail identifier
adapter_version(): string            // Returns semver adapter version    //
READ OPERATIONS   read_event(     tx_hash:      hex_string,     output_index:
integer | null   ): RailObject                        // Read and normalize a
raw rail event    read_block(     block_height: integer   ): RailObject[]
// Read all URIB-relevant events in a block    get_finality_status(
tx_hash: hex_string,     block_height: integer   ): FinalizationStatus
// UNCONFIRMED | CONFIRMING | FINALIZED    // WRITE OPERATIONS
publish_taproot_root(     taproot_root: hex_string,          // 32-byte TR
batch_id:     string               // UUID of the commitment batch   ):
RailAnchor                          // Returns the rail anchor for the
publication tx    publish_nullifier(     nullifier:    hex_string
// 32-byte nullifier value   ): RailAnchor    // MAPPING   to_canonical(
rail_object:  RailObject,     semantic_id:  string | null        // Optional:
pre-assigned SID   ): CanonicalObject                    // Layer 1 → Layer 2
from_canonical(     canonical_obj: CanonicalObject   ): RailObject
// Layer 2 → Layer 1 (for write operations)    // VALIDATION
verify_taproot_inclusion(     leaf_hash:    hex_string,     merkle_path:
MerklePathEntry[],     taproot_root: hex_string   ): boolean
// Returns true if leaf is in the Taproot tree    // ERROR HANDLING   // All
methods MUST throw typed errors from the URIB Error Code Registry (Section
11.7) }
```

4.4 Worked Examples

4.4.1 Example 1: Bitcoin UTXO → EVM ERC-20

Scenario: Alice holds 0.5 BTC in a UTXO on Bitcoin mainnet. She bridges this to an equivalent WBTC (Wrapped Bitcoin) ERC-20 balance on Ethereum mainnet.

Step 1: Raw Bitcoin UTXO (Layer 0)

```
// Bitcoin UTXO {   "txid":         "abc123...fed987",   "vout":         0,
"address":      "bc1qalice...xyz",   "value_sats":   50000000,          //
0.5 BTC = 50,000,000 satoshis   "block_height": 851200,   "script":
"0014...alice_pk_hash...",   "confirmations": 6 }
```

Step 2: Canonical Form (Layer 2)

```
// Canonical Object — BTC UTXO {   "canonical_id":  "utxo-a1b2c3d4-...",
"semantic_id":   "urn:urib:sid:asset:btc-utxo-v1",   "ordinal":       {
"rail_id": "btc-mainnet", "epoch": 4412, "sequence": 10001 },
"payload_hash":  "sha3hash_of_utxo_payload...",   "rail_anchors":  [{
"rail_id": "btc-mainnet", "block_height": 851200,
"tx_hash": "abc123...fed987", "output_index": 0 }],   "status":
"ACTIVE",   "metadata":      { "tags": ["ASSET"], "description": "0.5 BTC
UTXO" } }
```

Step 3: Bridge Commitment Issued

```
// Bridge Commitment — BTC → ETH {   "leaf_id":         "bridge-e5f6a7b8-
...",   "canonical_id":    "utxo-a1b2c3d4-...",   "rail_from":       "btc-
mainnet",   "rail_to":         "eth-mainnet",   "state_hash_from":
"sha3hash_of_btc_state...",   "state_hash_to":   "sha3hash_of_eth_state...",
// Computed from EVM target representation   "ordinal_from":    { "rail_id":
"btc-mainnet", "epoch": 4412, "sequence": 10001 },   "ordinal_to":      {
"rail_id": "eth-mainnet",  "epoch": 4412, "sequence": 5501  } }
```

Step 4: EVM Mapped Form (after bridge)

```
// Rail Object — Ethereum (WBTC ERC-20) {   "rail_id":       "eth-mainnet",
"block_height":  20048500,   "tx_hash":       "0xdef456...abc789",
"payload": {     "contract": "0x2260FAC5E5542a773Aa44fBCfeDf7C193bc2C599",
// WBTC     "to":       "0xAliceETHAddress",     "value":    "50000000"
// Preserved: 50,000,000 units (8 decimal WBTC = 0.5 BTC)   },
"finality_status": "FINALIZED" } // NOTE: The semantic_id remains identical:
urn:urib:sid:asset:btc-utxo-v1 // The WBTC representation is a rail-view of
the same canonical asset.
```

4.4.2 Example 2: XRPL Trust Line → Solana SPL Token

Scenario: Carol holds a USD trust line balance of 100 USD on XRPL. She bridges to a USDC SPL token balance on Solana.

```
// Step 1: XRPL Trust Line (Layer 0) {   "account":   "rCarol...XRPLaddress",
"currency":  "USD",   "issuer":    "rBitstampBankAddress...",   "balance":
"100",   "limit":     "1000",   "ledger_index": 85400200 }  // Step 2:
Canonical Object (Layer 2) {   "canonical_id":  "xrpl-trust-cc1d2e3f-...",
"semantic_id":   "urn:urib:sid:asset:usd-claim-v1",   "ordinal":       {
"rail_id": "xrpl-mainnet", "epoch": 4410, "sequence": 3301 },
"payload_hash":  "sha3hash_trustline...",   "rail_anchors":  [{ "rail_id":
"xrpl-mainnet", "block_height": 85400200, ... }],   "metadata":      {
"tags": ["ASSET", "CLAIM"], "description": "100 USD XRPL trust line" } }  //
Step 3: Mapped Solana SPL Form (after bridge) {   "rail_id":    "solana-
mainnet",   "slot":       290114000,   "tx_hash":    "SolanaSignature...abc",
"payload": {     "mint":
"EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v",  // USDC SPL mint
"token_account": "CarolSolanaAssociatedTokenAddress...",     "amount":
"100000000"    // 100 USDC × 10^6 decimals   },   "finality_status":
"FINALIZED" } // semantic_id preserved: urn:urib:sid:asset:usd-claim-v1 //
Precision note: XRPL uses 15-digit precision; SPL USDC uses 6 decimals.
Declared in Rail Map Registry.
```

4.5 Rail-Specific Quirks Reference

Rail	Finality Model	Address Format	Asset Model	URIB Adapter Notes
Bitcoin	Probabilistic; 6 blocks ≈ 60 min recommended	Bech32 (P2WPKH, P2TR)	UTXO (unspent transaction output)	TR published via OP_RETURN in P2TR tx. Nullifiers as OP_RETURN. Satoshi precision (8 decimals).
Ethereum	Deterministic; 2 epochs ≈ 12.8 min (post-Merge)	EIP-55 checksummed hex (0x...)	Account-based; ERC-20 balances; ERC-721 NFTs	Commitment Registry as verified smart contract. Events emitted for all commitments. Gas cost must be factored into bridge fee estimation.
XRPL	Deterministic; ~3–5 seconds (consensus rounds)	Base58Check (r...)	Native XRP + IOU trust lines + NFTokens	TR in AccountSet Memo. Trust line precision: 15 significant digits. Issuer dependency must be tracked in Rail Map Registry.
Solana	Optimistic; ~0.4s slot; ~32 slots for finality	Base58 (44 chars)	SPL tokens; PDAs for program accounts	URIB Solana Program (PDA-managed). High throughput but reorg risk within first few slots. Ordinal assignment deferred to finalized slots.

Rail	Finality Model	Address Format	Asset Model	URIB Adapter Notes
IPFS	Content-addressed; no native finality	CIDv1 (bafy...)	Immutable content; no ownership model	IPFS used as content storage layer only. Pinning service must be designated. CID published as rail anchor on a finality rail (e.g., Ethereum). IPFS not used as settlement rail.
HTTPS	None (stateless); depends on server	URL + path	REST resources; JSON payloads	HTTPS rails used for off-chain data attestations and webhook-based bridge notifications. All payloads MUST be signed with Ed25519. TLS 1.3 mandatory. Used as auxiliary rail; never as primary settlement rail.

---

5. Cross-Rail Invariants

> *This section formally defines all eight cross-rail invariants that every conformant URIB implementation must preserve at all times. Violation of any invariant constitutes a critical implementation defect.*

Invariant I-1: Canonical Uniqueness

Formal Statement: For all canonical objects O1, O2 in the URIB system: if O1 ≠ O2 (i.e., they differ in at least one field), then C-Hash(O1) ≠ C-Hash(O2).

Proof Sketch: The C-Hash is computed as SHA3-256 over the concatenation of canonical_id, ordinal bytes, payload_hash, and sorted rail anchor hashes. Since canonical_id is a UUID v5 (computed from SID + ordinal, which is globally unique by I-3), and SHA3-256 is collision-resistant under standard cryptographic assumptions, the probability of two distinct objects sharing a C-Hash is negligible (2^{-256}).

Violation Detection: At canonicalization time, the URIB registry performs a C-Hash lookup. If the computed C-Hash already exists in the registry for a different object, an `ERR_CHASH_COLLISION` error is raised and the new canonicalization is rejected.

Invariant I-2: Semantic Consistency

Formal Statement: For all canonical objects O and all rails R1, R2 on which O has a rail anchor: resolve_SID(SID(O), R1) = resolve_SID(SID(O), R2). That is, the SID of a canonical object resolves to the same semantic concept regardless of which rail is used for resolution.

Proof Sketch: The Semantic Graph is append-only and content-addressed. A SID is a stable URN that hashes the canonical concept definition — a definition that does not depend on any rail. Rail-

specific views are mappings of the semantic concept, not redefinitions of it. Since SID resolution is performed against the global Semantic Graph (not a rail-local index), rail context cannot alter the resolution outcome.

Violation Detection: During rail mapping (ϕ), the mapping function asserts that SID(O) == SID(O'). If this assertion fails, the mapping is rejected with `ERR_SEMANTIC_INCONSISTENCY`.

Invariant I-3: Ordinal Monotonicity

Formal Statement: For all canonical events E1, E2 on the same rail R: if E1 precedes E2 in causal order, then ordinal(E1).sequence < ordinal(E2).sequence. For all canonical events E1 on rail R1 and E2 on rail R2: if epoch(E1) < epoch(E2), then E1 causally precedes E2.

Proof Sketch: Sequence numbers are issued by the Ordinal Registry as a strictly-incrementing counter with atomic compare-and-swap guarantees. Epoch boundaries are synchronized across all rails via epoch anchor events. The vector clock mechanism ensures cross-rail causal ordering within an epoch.

Violation Detection: The Ordinal Registry maintains a per-rail high-water mark. Any ordinal assignment with sequence ≤ current high-water mark is rejected with `ERR_ORDINAL_NON_MONOTONIC`. Epoch boundary violations trigger `ERR_EPOCH_SEQUENCE_VIOLATION`.

Invariant I-4: Identity Continuity

Formal Statement: For all ThreadZero identities TZ and all thread events E on TZ: there exists a valid continuity proof path(E, genesis(TZ)) consisting of a sequence of events each referencing the parent_hash of its predecessor, terminating at the genesis event of TZ. This path must be verifiable against the published thread Merkle root.

Proof Sketch: Thread events are append-only and each references `parent_hash` of the preceding event. The causal chain from any event back to genesis is therefore unbroken by construction. The

Merkle tree over thread events provides efficient path verification. Rail transitions are recorded as RAIL_BRIDGE events on the thread, maintaining the chain across rail changes.

Violation Detection: Any thread event whose `parent_hash` does not match the hash of the most recently accepted event on the same thread is rejected with `ERR_THREAD_CONTINUITY_BROKEN`.

Invariant I-5: Non-Duplication

Formal Statement: For all canonical objects O: at any point in time T, O.status == ACTIVE on at most one rail. If O is BRIDGING, it is in ACTIVE state on neither the source nor the destination rail (it is locked). O may exist as a NULLIFIED record on multiple rails, but may not be simultaneously ACTIVE on more than one.

Proof Sketch: When a bridge operation initiates, the source-rail Rail Adapter publishes a nullifier for the source representation of O before the destination rail activates O'. The nullifier is globally visible and prevents re-activation on the source rail. The bridge commitment tracks both state_hash_from and state_hash_to, ensuring that only one can be ACTIVE at any given time.

Violation Detection: The URIB Nullifier Registry checks for existing active states before any bridge initiation. Detection of an ACTIVE object on more than one rail triggers `ERR_DUPLICATION_DETECTED` and initiates an emergency reconciliation procedure.

Invariant I-6: Bridge Soundness

Formal Statement: A bridge commitment BC is valid if and only if: (a) canonical_hash(BC.state_hash_from) is a valid C-Hash in the URIB registry, (b) canonical_hash(BC.state_hash_to) is a valid C-Hash in the URIB registry, (c) the ordinal window [ordinal_from, ordinal_to] lies entirely within the finality window of both participating rails at the time of commitment creation, and (d) BC carries valid witness signatures from all required signers.

Proof Sketch: Each field of validity is checked independently. C-Hash validity is verified against the registry. Ordinal window validity is checked against per-rail finality window parameters (see Section

8). Witness signatures are verified against Ed25519 public keys registered in the ThreadZero of each signer.

Violation Detection: Bridge commitment validators run all four checks before accepting a commitment. Any check failure produces a typed error from the ERR_BRIDGE_* namespace. Invalid commitments are rejected and logged for audit.

Invariant I-7: Settlement Equivalence

Formal Statement: For rails R1, R2 and canonical object O: if there exists a FINALIZED bridge commitment BC linking O on R1 to O' on R2, and the Taproot Root containing BC's commitment leaf is anchored on both R1 and R2, then settlement_finalized(O, R1) ⟺ settlement_finalized(O', R2).

Proof Sketch: Finalization of BC requires that both R1 and R2 have confirmed the Taproot Root containing BC's leaf past their respective finality windows. The Taproot Root's Merkle structure allows any party to verify BC's inclusion with a O(log n) proof. The bi-directional anchoring ensures neither side can be unilaterally reversed after FINALIZED status is reached.

Violation Detection: Settlement equivalence validators periodically audit all FINALIZED commitments by verifying Taproot Root anchors on both rails. Discrepancies trigger `ERR_SETTLEMENT_EQUIVALENCE_VIOLATED` and initiate dispute resolution.

Invariant I-8: Semantic Losslessness

Formal Statement: For all canonical objects O and rail mappings ϕ(O, R_src, R_dst) → O': semantic_tags(O') ⊇ semantic_tags(O) and SID(O') = SID(O). No semantic tag present on O may be absent on O'. Additional tags may be added by ϕ if the destination rail supports them, but no tag removal is permitted.

Proof Sketch: Rail mapping functions are specified in the Rail Map Registry with a declared `tag_preservation` guarantee. The mapping function implementation is required to copy all source semantic tags to the destination object before returning. A post-mapping assertion verifies tag set containment.

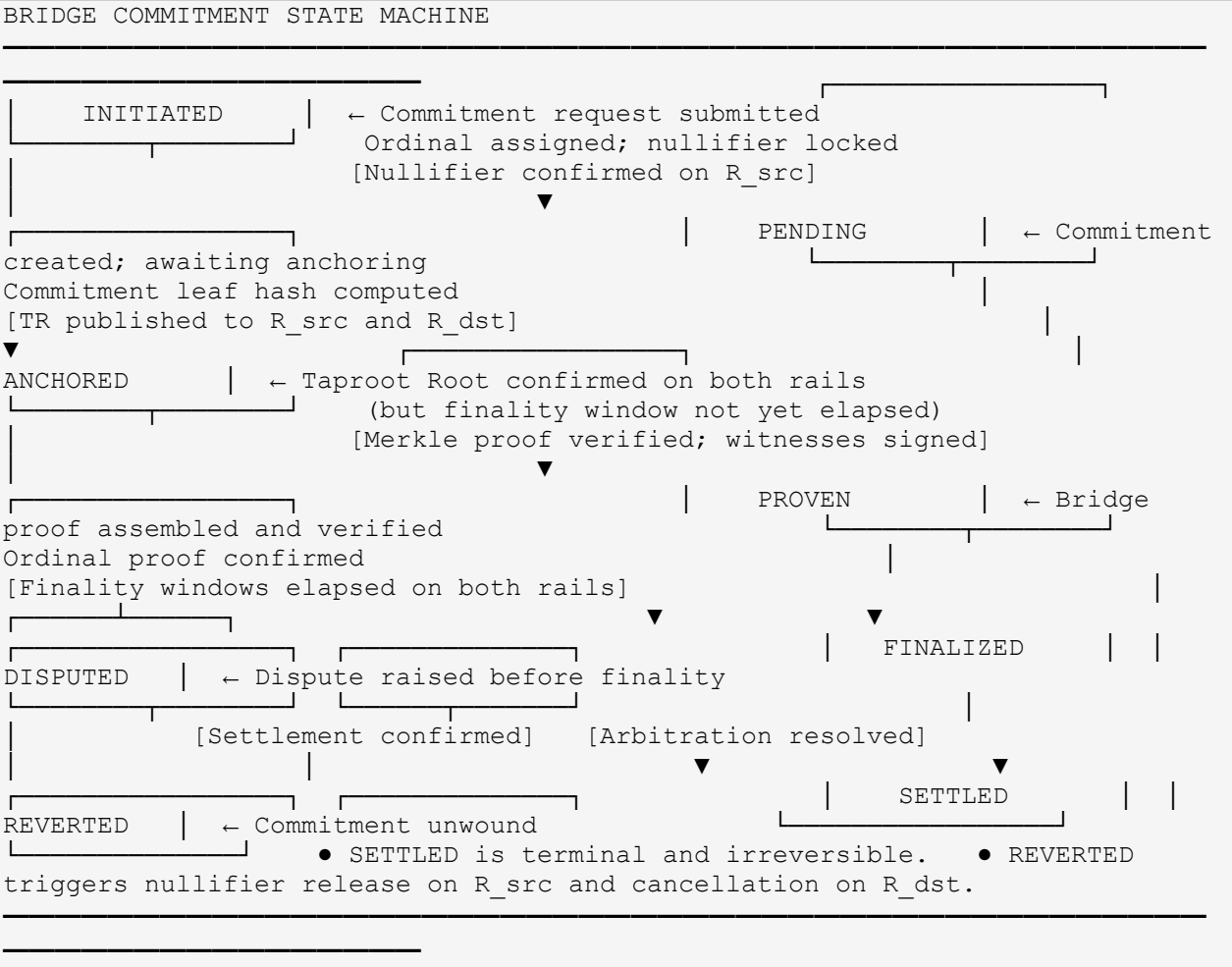
Violation Detection: Post-mapping validation computes `semantic_tags(O) \ semantic_tags(O')`. If this set difference is non-empty, the mapping is rejected with `ERR_SEMANTIC_LOSSLESSNESS_VIOLATED`.

6. Bridge Commitments

> *This section fully specifies the lifecycle, schema, issuance, verification, aggregation, and security properties of URIB Bridge Commitments.*

6.1 Bridge Commitment Lifecycle

A Bridge Commitment progresses through the following state machine:



6.2 Bridge Commitment Data Structure

```
// Bridge Commitment — Full Schema v1.0 {   "schema_version":       "1.0",
"commitment_id":        string,           // UUID v5 (sha1_ns + canonical_id
+ ordinal_from)   "canonical_id":         string,           // CID of the
object being bridged   "semantic_id":          string,           // SID (must
match canonical object's SID)    "rail_from":           string,           //
Source rail identifier   "rail_to":              string,           //
Destination rail identifier   "address_from":         string,           //
Source rail address (native format)   "address_to":           string,
// Destination rail address (native format)    "state_hash_from":
hex_string,       // SHA3-256 of object state on rail_from before bridge
"state_hash_to":        hex_string,       // SHA3-256 of object state on
rail_to after bridge   "payload_from":         object,           // Canonical
payload on source rail   "payload_to":           object,           //
Canonical payload on destination rail    "ordinal_from":
OrdinalNamespace, // Ordinal of bridge initiation event   "ordinal_to":
OrdinalNamespace, // Ordinal of bridge completion event (set at PROVEN)
"epoch":                integer,          // Epoch in which this commitment
was issued    "taproot_root":         hex_string | null,// TR of the batch
containing this commitment (set at ANCHORED)   "leaf_hash":
hex_string,       // SHA3-256 of this commitment's leaf serialization
"merkle_path":          MerklePathEntry[] | null,  // Merkle proof (set at
PROVEN)    "nullifier":            hex_string,       // Published nullifier
for source-rail object   "nullifier_anchor":     RailAnchor,       // Rail
anchor where nullifier was published    "status":               enum {
"INITIATED", "PENDING", "ANCHORED",                                "PROVEN",
"FINALIZED", "SETTLED",                                "DISPUTED", "REVERTED"
},    "witness_sigs":         [WitnessSig],     // Required witness
signatures   "initiator_thread_id":  string,           // ThreadZero ID of
the initiating identity   "created_at":           ISO8601_string,
"anchored_at":          ISO8601_string | null,   "proven_at":
ISO8601_string | null,   "finalized_at":         ISO8601_string | null,
"settled_at":           ISO8601_string | null,    "finality_window_src":
integer,          // Expected finality window seconds for rail_from
"finality_window_dst":  integer,          // Expected finality window seconds
for rail_to   "dispute_deadline":     ISO8601_string,   // Latest time a
dispute may be raised    "metadata":             object | null     //
Arbitrary extension fields }
```

6.3 Commitment Issuance

Authorization: A bridge commitment may only be issued by a URIB-authorized entity whose ThreadZero is registered in the URIB Identity Registry. The issuer must hold the signing key for the source-rail address (`address_from`) and provide a witness signature over the commitment payload.

Issuance Conditions:

23. The canonical object MUST be in ACTIVE status on the source rail.
24. No existing non-SETTLED or non-REVERTED commitment may exist for the same canonical_id.
25. A valid ordinal must be obtained from the Ordinal Registry.
26. The nullifier for the source object must be computed and ready for publication.
27. The issuer's ThreadZero must be in ACTIVE status (not TOMBSTONED).

6.4 Commitment Proof Verification Algorithm

```
// Commitment Proof Verification — Step-by-Step  function
verify_bridge_commitment(commitment: BridgeCommitment, proof: BridgeProof) →
boolean:    // Step 1: Verify commitment schema integrity   assert
commitment.schema_version == "1.0"   assert commitment.canonical_id is valid
UUID   assert commitment.semantic_id matches SID format    // Step 2: Verify
C-Hash of both state hashes   obj_from =
registry.get_canonical_object(commitment.state_hash_from)   assert
obj_from.c_hash == compute_chash(obj_from)   obj_to =
registry.get_canonical_object(commitment.state_hash_to)   assert
obj_to.c_hash == compute_chash(obj_to)    // Step 3: Verify Semantic
Losslessness (I-8)   assert obj_to.semantic_id == obj_from.semantic_id
assert semantic_tags(obj_to) ⊇ semantic_tags(obj_from)    // Step 4: Verify
Ordinal validity and monotonicity (I-3)   assert
ordinal_registry.is_valid(commitment.ordinal_from)   assert
ordinal_registry.is_valid(commitment.ordinal_to)   assert
epoch(commitment.ordinal_from) ≤ epoch(commitment.ordinal_to)    // Step 5:
Verify Taproot inclusion   assert verify_merkle_proof(     leaf_hash    =
commitment.leaf_hash,     merkle_path  = commitment.merkle_path,     root
= commitment.taproot_root   )    // Step 6: Verify Taproot Root anchored on
both rails   anchor_src =
rail_adapter(commitment.rail_from).get_taproot_anchor(commitment.taproot_root
)   anchor_dst =
rail_adapter(commitment.rail_to).get_taproot_anchor(commitment.taproot_root)
assert anchor_src.finality_status == "FINALIZED"   assert
anchor_dst.finality_status == "FINALIZED"    // Step 7: Verify finality
windows elapsed   now = current_utc_time()   assert (now -
anchor_src.timestamp) >= commitment.finality_window_src   assert (now -
anchor_dst.timestamp) >= commitment.finality_window_dst    // Step 8: Verify
nullifier published and confirmed   assert
nullifier_registry.is_published(commitment.nullifier)   assert
nullifier_registry.get_anchor(commitment.nullifier).finality_status ==
"FINALIZED"    // Step 9: Verify witness signatures   for sig in
```

```
commitment.witness_sigs:     pk =
identity_registry.get_public_key(sig.signer_id)     assert ed25519_verify(pk,
sig.signature, canonical_serialize(commitment))    // Step 10: Verify no
active dispute   assert
dispute_registry.get_disputes(commitment.commitment_id) == []    return true
// Commitment is valid and fully proven
```

6.5 Commitment Aggregation

Multiple bridge commitments are batched into a single Taproot Root (TR) for efficiency. The batch lifecycle is:

28. **Batch collection window:** The Commitment Batcher collects pending commitments over a configurable time window (default: 60 seconds or 256 commitments, whichever comes first).
29. **Leaf hash computation:** For each commitment in the batch, compute `leaf_hash = SHA3-256(canonical_serialize(commitment_leaf))`.
30. **Taproot Root computation:** Apply the Taproot Root algorithm (Section 3.6.3) over all leaf hashes.
31. **TR publication:** Publish the TR to all rails participating in the batch (Section 3.6.6).
32. **Merkle path distribution:** For each commitment, compute and distribute its Merkle path against the TR. Each commitment holder uses this path to independently verify their commitment's inclusion.

6.6 Commitment Revocation and Dispute Resolution

A bridge commitment may be disputed before it reaches FINALIZED status. Disputes are raised by submitting a **Dispute Notice** to the URIB Dispute Registry before the `dispute_deadline`. Grounds for dispute include:

- **Invalid nullifier:** The source nullifier was not published or is invalid.
- **State hash mismatch:** The destination state hash does not correspond to the canonical object's actual state on the destination rail.
- **Witness fraud:** A witness signature is forged or the signer was not authorized.
- **Ordinal conflict:** The ordinal assigned to the commitment conflicts with another event.
- **Finality window violation:** The commitment was finalized before the finality window elapsed.

Upon a valid dispute, the commitment transitions to DISPUTED status. An arbitration process (which may be on-chain smart contract arbitration or off-chain URIB arbitration panel decision, depending on rail configuration) resolves the dispute. If the dispute is upheld, the commitment transitions to REVERTED and the nullifier is released.

6.7 Security Considerations for Bridge Commitments

- **Replay Attack Prevention:** Each commitment carries a globally unique `commitment_id` (UUID v5 computed from canonical_id + ordinal_from). The Commitment Registry rejects any commitment whose ID already exists. Nullifiers are single-use and permanently recorded.
- **Double-Spend Prevention:** The nullifier mechanism ensures that once a source-rail object is committed to a bridge, it cannot be spent on the source rail. The nullifier is published before the destination object is activated.
- **Commitment Front-Running:** To prevent front-running (an adversary intercepting and submitting a commitment first), commitments include a commitment hash (commit-reveal scheme) that must be revealed within the dispute window. The commit hash is submitted first; the full commitment is revealed only after the commit hash is confirmed.

- **Commitment Expiry:** Commitments that remain in PENDING or ANCHORED status beyond 3× the maximum finality window of participating rails are automatically expired and the nullifier released.

7. Identity Continuity

> *This section specifies the full set of identity continuity guarantees provided by URIB, including ThreadZero semantics, the fork prohibition, continuity proof algorithm, cross-rail identity handoff, revocation, key rotation, and a worked example.*

7.1 Identity Continuity Guarantees

URIB provides the following identity continuity guarantees for all ThreadZero identities:

33. **Immutable Origin:** The genesis event of a ThreadZero is cryptographically committed on at least one rail and cannot be altered.
34. **Causal Completeness:** Every state change to an identity is recorded as a Thread Event with a valid parent_hash and ordinal, forming an unbroken causal chain.
35. **Cross-Rail Persistence:** An identity that crosses rails records a RAIL_BRIDGE event on its thread, maintaining continuity through the rail transition.
36. **Key Rotation Safety:** An identity may rotate its cryptographic keys via a KEY_ROTATION thread event without losing continuity. The new key is committed on the thread and the old key is revoked.
37. **Non-Mergeability:** Two ThreadZero identities can never be merged into one. They may be related by RELATIONSHIP edges in the Semantic Graph, but they remain distinct identity threads.

7.2 Fork Prohibition

Derived threads may branch from ThreadZero, but they **MUST NOT merge** with each other or with the parent ThreadZero. The fork prohibition is enforced as follows:

- A Thread Event is only valid if it references exactly one `parent_hash` — the most recent event on the same thread. No event may reference parent hashes from two different threads.
- The URIB Identity Registry rejects any thread event whose `derived_from` field differs from the thread's established parent thread ID.
- Merging of two identities is achieved exclusively through RELATIONSHIP edges in the Semantic Graph — never by collapsing two threads into one.

7.3 Continuity Proof Algorithm

```
// Continuity Proof Algorithm // Given: two thread events E1, E2 on the same
ThreadZero TZ // Produce: proof that E1 and E2 share the same ThreadZero
origin  function prove_continuity(E1: ThreadEvent, E2: ThreadEvent, TZ:
ThreadZero):    // Step 1: Verify both events belong to the same thread
assert E1.thread_id == TZ.thread_id   assert E2.thread_id == TZ.thread_id
// Step 2: Retrieve the full ordered event chain from genesis to
max(E1.ordinal, E2.ordinal)   chain =
thread_registry.get_event_chain(TZ.thread_id, from_genesis=True,
to_ordinal=max(E1.ordinal, E2.ordinal))    // Step 3: Verify chain continuity
(each event's parent_hash matches predecessor)   for i in range(1,
len(chain)):     assert chain[i].parent_hash == SHA3-
256(canonical_serialize(chain[i-1]))    // Step 4: Verify genesis event
parent_hash is the null hash   assert chain[0].parent_hash == "0x" + "00" *
32   assert chain[0].event_type == "GENESIS"    // Step 5: Locate E1 and E2
in the chain   idx_E1 = find_in_chain(chain, E1.event_ordinal)   idx_E2 =
find_in_chain(chain, E2.event_ordinal)   assert idx_E1 is not None   assert
idx_E2 is not None    // Step 6: Compute Merkle path from each event to the
chain root   chain_merkle_root = compute_merkle_root([SHA3-
256(canonical_serialize(e)) for e in chain])   path_E1 =
compute_merkle_path(chain, idx_E1)   path_E2 = compute_merkle_path(chain,
idx_E2)    // Step 7: Return the proof   return ContinuityProof {
thread_id:    TZ.thread_id,     genesis_hash: SHA3-
256(canonical_serialize(chain[0])),     event1_hash:  SHA3-
256(canonical_serialize(E1)),     event2_hash:  SHA3-
256(canonical_serialize(E2)),     chain_root:   chain_merkle_root,
path_E1:      path_E1,     path_E2:      path_E2   }
```

7.4 Cross-Rail Identity Handoff

When an identity moves from one rail to another (e.g., a user migrates their primary activity from Ethereum to XRPL), the following procedure maintains identity continuity:

38. On the current rail (R1), append a `RAIL_BRIDGE` Thread Event to the ThreadZero, specifying `target_rail = R2` and the new rail address.
39. Obtain an ordinal from the Ordinal Layer for this RAIL_BRIDGE event.
40. Publish the RAIL_BRIDGE Thread Event to R1 and obtain a rail anchor.
41. On the destination rail (R2), publish a corresponding `RAIL_BRIDGE` Thread Event referencing the R1 anchor, the same ordinal, and the new key pair (if key rotation is also occurring).
42. Issue a Bridge Commitment for the identity's primary canonical object (its root IDENTITY-tagged object).
43. After the bridge commitment is FINALIZED, the identity is considered operationally resident on R2, with full continuity proof chain including both the R1 and R2 RAIL_BRIDGE events.

7.5 Identity Revocation (Tombstone Pattern)

A ThreadZero identity is deactivated by appending a TOMBSTONE Thread Event. The TOMBSTONE event:

- Is the final event on the thread — no further events may be appended after TOMBSTONE.
- Must be signed by the identity's current active signing key with a witness signature.
- May optionally include a `successor_thread_id` pointing to a different ThreadZero that succeeds the tombstoned identity (for identity migration or legal succession scenarios).
- Does NOT delete the thread history. All prior events remain accessible for audit and proof purposes.
- Causes all active derived threads to be automatically revoked (REVOCATION events are appended by the URIB system).

7.6 Key Rotation and Continuity

Cryptographic key rotation is performed without breaking identity continuity via the KEY_ROTATION Thread Event:

44. Generate a new Ed25519 key pair `(sk_new, pk_new)`.
45. Construct the KEY_ROTATION payload: `{ "old_pk": pk_current_hex, "new_pk": pk_new_hex, "rotation_reason": string }`.
46. Sign the KEY_ROTATION Thread Event with **both** the current key `sk_current` AND the new key `sk_new`. Both signatures are included in `witness_sigs`. This dual-signature requirement prevents unauthorized key rotation.
47. Publish the KEY_ROTATION event to the current primary rail.
48. After the event is FINALIZED on the rail, securely destroy `sk_current`.
49. All future thread events are signed exclusively with `sk_new`.

7.7 Worked Example — Identity Across Three Rails

```
IDENTITY CONTINUITY EXAMPLE — ETH → BTC → XRPL
═══════════════════════════════════════════════════════════════════════════

──────────────────────────  Identity Entity E — ThreadZero ID:
urn:urib:thread:genesis_hash_abc123  STEP 1 — Genesis on Ethereum (Ordinal:
eth-mainnet, epoch 4400, seq 1)   ThreadEvent {     event_type:   "GENESIS",
thread_id:    "urn:urib:thread:genesis_hash_abc123",     parent_hash:
"0x0000...0000",  // null predecessor     rail_anchor:  { rail_id: "eth-
mainnet", block_height: 19900000, tx: "0xGEN..." }     witness_sig:
[Ed25519(sk_eth1, genesis_bytes)]   }   → C-Hash established. Identity ACTIVE
on eth-mainnet.  STEP 2 — Bridge to Bitcoin (Ordinal: eth-mainnet, epoch
4405, seq 88)   ThreadEvent {     event_type:   "RAIL_BRIDGE",
parent_hash:  SHA3-256(genesis_event_bytes),     payload:      { target_rail:
"btc-mainnet", btc_address: "bc1qE..." },     rail_anchor:  { rail_id: "eth-
mainnet", block_height: 19940000 }     witness_sig:  [Ed25519(sk_eth1,
rail_bridge_1_bytes)]   }   → Bridge Commitment BC-001 issued (ETH→BTC).
Nullifier published on ETH.  STEP 3 — RAIL_BRIDGE confirmed on Bitcoin
(Ordinal: btc-mainnet, epoch 4405, seq 1)   ThreadEvent {     event_type:
"RAIL_BRIDGE",     parent_hash:  SHA3-256(rail_bridge_eth_bytes),  // ←
continues chain     payload:      { source_rail: "eth-mainnet", eth_anchor:
"0xGEN..." },     rail_anchor:  { rail_id: "btc-mainnet", block_height:
850000 }     witness_sig:  [Ed25519(sk_btc1, rail_bridge_2_bytes)]   }   →
BC-001 now FINALIZED. Identity ACTIVE on btc-mainnet.   → Continuity proof:
genesis_hash → rail_bridge_eth → rail_bridge_btc ✓  STEP 4 — Key Rotation on
Bitcoin (Ordinal: btc-mainnet, epoch 4408, seq 5)   ThreadEvent {
event_type:   "KEY_ROTATION",     parent_hash:  SHA3-
256(rail_bridge_btc_bytes),     payload:      { old_pk: pk_btc1_hex, new_pk:
pk_btc2_hex },     witness_sigs: [Ed25519(sk_btc1,...), Ed25519(sk_btc2,...)]
// Dual-signed   }   → sk_btc1 destroyed. sk_btc2 now active.  STEP 5 —
Bridge to XRPL (Ordinal: btc-mainnet, epoch 4412, seq 10)   ThreadEvent {
event_type:   "RAIL_BRIDGE",     parent_hash:  SHA3-256(key_rotation_bytes),
payload:      { target_rail: "xrpl-mainnet", xrpl_address: "rEidentity..." },
witness_sig:  [Ed25519(sk_btc2, rail_bridge_3_bytes)]   }   → Bridge
Commitment BC-002 issued (BTC→XRPL).  STEP 6 — RAIL_BRIDGE confirmed on XRPL
(Ordinal: xrpl-mainnet, epoch 4412, seq 1)   ThreadEvent {     event_type:
"RAIL_BRIDGE",     parent_hash:  SHA3-256(rail_bridge_btc2_bytes),
witness_sig:  [Ed25519(sk_xrpl1, rail_bridge_4_bytes)]   }   → BC-002
FINALIZED. Identity ACTIVE on xrpl-mainnet.   → Full Continuity Proof Chain:
genesis(ETH) → bridge(ETH→BTC) → bridge_recv(BTC) → key_rot(BTC)
→ bridge(BTC→XRPL) → bridge_recv(XRPL)       All 6 events verifiable via
Merkle paths against published thread root. ✓
═══════════════════════════════════════════════════════════════════════════
──────────────────────────
```

8. Settlement Equivalence

> *This section defines the formal conditions for settlement equivalence, the per-rail finality windows, the settlement proof structure, the cross-rail settlement workflow, and failure mode handling.*

8.1 Formal Definition

Settlement equivalence between rails R1 and R2 for canonical object O is formally defined as:

> *Settlement on R1 for canonical object O is equivalent to settlement on R2 for the corresponding canonical object O' if and only if: (a) there exists a Bridge Commitment BC in status FINALIZED linking O on R1 to O' on R2; (b) the Taproot Root containing BC's commitment leaf is confirmed on R1 past R1's finality window; (c) the same Taproot Root is confirmed on R2 past R2's finality window; and (d) the nullifier for O on R1 is confirmed on R1 past R1's finality window.*

Settlement equivalence is symmetric: if settlement on R1 is equivalent to settlement on R2, then settlement on R2 is equivalent to settlement on R1 (given the same FINALIZED BC).

8.2 Finality Window Reference

Rail	Finality Model	Finality Window	Confirmation Metric	URIB Conservative Window
Bitcoin	Probabilistic (Nakamoto)	~60 minutes	6 blocks (each ~10 min)	90 minutes (9 blocks)
Ethereum	Deterministic (Casper FFG)	~12.8 minutes	2 epochs (each ~6.4 min)	15 minutes
XRPL	Deterministic (RPCA)	~3–5 seconds	1 consensus round	30 seconds (safety buffer)
Solana	Optimistic (Tower BFT)	~13 seconds	32 slots (~0.4s each)	30 seconds
IPFS	N/A (content-addressed)	N/A	Not applicable	Depends on anchoring rail
HTTPS	None (stateless)	N/A	Not applicable	Never used as settlement rail

IMPORTANT — Conservative Windows

URIB implementations MUST use the conservative finality window values shown above, not the nominal values. The conservative window adds a safety buffer to account for network propagation delays, temporary forks (Bitcoin), and clock drift. Systems requiring stricter finality guarantees may extend these windows further; they MUST NOT shorten them.

8.3 Settlement Proof Structure

```
// Settlement Proof — Complete structure SettlementProof = {   "proof_id":
string,         // UUID v5   "canonical_id":          string,         // CID
of the settled object   "bridge_commitment":     BridgeCommitment, // Full
FINALIZED commitment   "taproot_root":          hex_string,     // The 32-
byte TR   "rail_anchor_R1": {                      // Where TR was published
on source rail     "rail_id":       string,     "block_height":  integer,
"tx_hash":       hex_string,     "confirmed_at":  ISO8601_string,
"finality_status": "FINALIZED"   },   "rail_anchor_R2": {
// Where TR was published on destination rail     "rail_id":       string,
"block_height":  integer,     "tx_hash":       hex_string,
"confirmed_at":  ISO8601_string,     "finality_status": "FINALIZED"   },
"ordinal_proof":         OrdinalProof,   // Proof of ordinal validity
"nullifier_proof": {     "nullifier":     hex_string,     "anchor":
RailAnchor,     "finality_status": "FINALIZED"   },   "finality_attestation":
{     "attestor_thread_id":  string,         // ThreadZero ID of the attestor
"attestation_sig":     hex_string,     // Ed25519 sig over proof hash
"attested_at":         ISO8601_string   },   "generated_at":
ISO8601_string }
```

8.4 Cross-Rail Settlement Workflow

50. **Initiation:** The initiating identity submits a bridge request specifying canonical_id, rail_from, rail_to, address_from, address_to, and amount (if ASSET).
51. **Ordinal Assignment:** The Ordinal Layer assigns an ordinal for the BRIDGE_INIT event.
52. **Commitment Computation:** state_hash_from and state_hash_to are computed. A Bridge Commitment is constructed with status INITIATED.
53. **Nullifier Publication:** The source Rail Adapter publishes the nullifier to rail_from. Commitment transitions to PENDING.
54. **Batch Assembly:** The Commitment Batcher collects this commitment and others into a batch.
55. **Taproot Root Computation:** TR is computed over the batch's sorted commitment leaf hashes.
56. **TR Publication:** TR is published to rail_from and rail_to. Commitment transitions to ANCHORED.
57. **Merkle Proof Verification:** Commitment holder verifies the Merkle path from their commitment leaf to the TR.
58. **Witness Signature Collection:** Required witnesses sign the commitment. Commitment transitions to PROVEN.

59. **Finality Window Wait:** System waits for both rails to confirm the TR past their conservative finality windows.
60. **Finality Verification:** Both rail anchors are verified as FINALIZED. Commitment transitions to FINALIZED.
61. **Ordinal Completion:** ordinal_to is assigned for the BRIDGE_COMPLETE event.
62. **Destination Activation:** The destination Rail Adapter activates the canonical object on rail_to.
63. **Settlement Proof Generation:** A complete SettlementProof is assembled and stored.
64. **Settlement:** Commitment transitions to SETTLED. The operation is complete and irrevocable.

8.5 Partial Settlement and Optimistic Settlement

Partial Settlement: Occurs when the destination rail (rail_to) confirms the TR before the source rail (rail_from). In this case, the commitment remains in ANCHORED status for rail_from until its finality window elapses. The destination object is NOT activated until both rails achieve FINALIZED status for the TR.

Optimistic Settlement: For low-value, high-trust operations, implementations MAY activate the destination object upon reaching ANCHORED status (before FINALIZED), subject to the following constraints:

- The optimistic activation MUST be clearly flagged as `settlement_status: "OPTIMISTIC"`.
- The object's spend or transfer on the destination rail MUST be blocked until FINALIZED is reached.
- If the commitment is reverted, the destination activation is reversed and the nullifier on rail_from is released.
- The risk of optimistic settlement is borne entirely by the operator, not by URIB protocol.

8.6 Settlement Failure Modes and Remediation

Failure Mode	Trigger	Remediation
TR publication failure on rail_from	Rail_from is congested or temporarily unavailable	Retry TR publication up to 3× with exponential backoff. If all fail, revert commitment.
TR publication failure on rail_to	Rail_to is congested or temporarily unavailable	Retry TR publication. Commitment remains in PENDING until TR published on both rails.
Finality regression (Bitcoin reorg)	A confirmed block is orphaned, un-confirming the TR anchor	Wait for re-confirmation. If anchor does not re-confirm within 2× finality window, revert.
Witness signature timeout	Required witnesses do not sign within the dispute_deadline	Treat as revert. Nullifier released. Commitment expires.
Rail bridge contract failure	On-chain commitment contract reverts (Ethereum)	Retry with higher gas limit. If persistent, escalate to off-chain arbitration.
Ordinal conflict	Two events issued with same ordinal	Conflict resolution via causal ordering. Lower-entropy hash tiebreaker. Ordinal Registry updated.

9. Security Model

> *This section defines the trust assumptions, threat model, cryptographic primitives, key management recommendations, and auditing requirements for conformant URIB implementations.*

9.1 Trust Assumptions Per Layer

Layer	Trusted Components	Trust Basis
Layer 0 (Physical Rail)	Rail consensus mechanism	Each rail's native security model (PoW, PoS, RPCA, etc.)
Layer 1 (Rail Normalization)	Rail Adapter implementation	Code-level trust; adapter must be audited and version-pinned
Layer 2 (Canonical Form)	SHA3-256 collision resistance	Cryptographic assumption; NIST-standardized
Layer 3 (Semantic)	Semantic Graph integrity	Content-addressed DAG; append-only; hash-verified
Layer 4 (ThreadZero)	Ed25519 signature security; private key custody	Cryptographic assumption + key management hygiene
Layer 5 (Taproot)	Merkle tree collision resistance; SHA3-256	Cryptographic assumption
Layer 6 (Ordinal)	Ordinal Registry availability and integrity	Distributed or replicated registry; threshold consensus

9.2 Threat Model

- **Rail Compromise:** If a rail is 51%-attacked or undergoes deep reorg, commitments anchored on that rail may be invalidated. Mitigation: use conservative finality windows; monitor rail health; fall back to multi-rail anchoring for high-value commitments.
- **Commitment Forgery:** An adversary attempts to construct a valid-looking Bridge Commitment without authorization. Mitigation: Ed25519 witness signatures verified against registered public keys; commitment IDs are UUID v5 computed from canonical inputs; C-Hash collision resistance.
- **Ordinal Collision:** An adversary attempts to issue two events with the same ordinal to create ambiguity. Mitigation: Ordinal Registry uses atomic compare-and-swap; duplicate ordinals are rejected at registration time; ordinal proofs are verifiable independently.
- **Semantic Spoofing:** An adversary attempts to issue a canonical object with a fraudulent SID, implying it represents a known semantic concept it does not. Mitigation: SIDs are computed from the canonical concept definition hash; re-creating an existing SID requires the same concept definition bytes; Semantic Graph nodes are content-addressed.
- **Identity Hijacking:** An adversary attempts to append events to a ThreadZero without holding the signing key. Mitigation: all thread events require a valid Ed25519 signature from the current active key; the key is recorded in the thread itself via KEY_ROTATION events; unauthorized events are rejected by thread validators.
- **Nullifier Withholding:** A bridge initiator publishes the bridge commitment but withholds the nullifier, allowing them to use the object on both rails. Mitigation: destination object activation is conditional on confirmed nullifier publication on rail_from; the bridge commitment remains in PENDING until the nullifier is confirmed.

9.3 Cryptographic Primitives

Primitive	Algorithm	Usage	Security Level
Hash function	SHA3-256 (FIPS 202)	C-Hash, payload_hash, leaf_hash, Taproot Root	128-bit
Digital signatures	Ed25519 (RFC 8032)	Witness signatures, thread event signing, commitment attestation	128-bit (curve25519)
Merkle trees	Binary Merkle (SHA3-256 nodes)	Taproot commitment tree, continuity proof paths	Same as hash function
Key derivation	HKDF-SHA3-256	Derived thread key derivation from parent seed	128-bit
UUID generation	UUID v5 (SHA-1 namespace)	canonical_id, commitment_id, leaf_id	N/A (identifier only)
ZK proofs (optional)	Groth16 or PLONK (SNARK)	Privacy-preserving ordinal proofs; selective commitment revelation	128-bit (pairing-based)

9.4 Key Management Recommendations

- ThreadZero signing keys MUST be stored in hardware security modules (HSMs) or equivalent secure enclaves for production deployments.
- Key rotation MUST be performed at least annually or immediately upon suspected compromise.
- Backup keys MUST be created at genesis and stored in geographically-distributed cold storage.
- No signing key MUST be shared between ThreadZero and any derived thread. Each thread MUST have its own independent key pair.
- Multi-signature threshold schemes (e.g., 3-of-5 Ed25519 multisig) are STRONGLY RECOMMENDED for high-value identities and organizational ThreadZeros.
- Key material MUST be zeroized from memory immediately after use.

9.5 Auditing and Monitoring Requirements

- All canonical object state transitions MUST be logged with timestamps, actor thread IDs, and cryptographic proofs.
- All bridge commitment state transitions MUST be logged and the log must be append-only and tamper-evident.
- Ordinal Registry state root MUST be published to at least two rails every epoch.
- Cross-rail invariant checks MUST be run by automated monitors at each epoch boundary. Violations MUST trigger alerts within 60 seconds.
- Settlement proof archives MUST be retained for a minimum of 7 years and MUST be reproducible from on-chain data.
- Cryptographic agility: implementations MUST include a migration path for upgrading SHA3-256 and Ed25519 to post-quantum alternatives when NIST standards are finalized.

10. Integration Guide for Developers

> *This section provides a practical, step-by-step guide for developers integrating systems with the URIB bridge stack, including a Rail Adapter implementation guide, canonical object creation, bridge commitment registration, invariant verification, and common errors.*

10.1 Integration Checklist

65. Read this specification in its entirety before beginning implementation.
66. Identify all rails your system will participate in and verify Rail Adapters exist for each.
67. Deploy or connect to the URIB Canonical Registry, Ordinal Registry, Nullifier Registry, and Semantic Graph.
68. Implement or configure a Rail Adapter for each participating rail (Section 4.3).
69. Create your organization's ThreadZero identity (one per legal entity or system component).
70. Implement the Canonicalization Algorithm (Section 3.3.2) in your primary language.
71. Implement C-Hash computation (Section 3.3.3 and Appendix A).
72. Implement Taproot Root computation (Section 3.6.3 and Appendix B).
73. Implement the Bridge Commitment lifecycle state machine (Section 6.1).
74. Implement all eight cross-rail invariant checks (Section 5) as runtime assertions.
75. Run the full conformance checklist (Appendix F) before claiming URIB conformance.
76. Submit to a third-party security audit before production deployment.

10.2 Rail Adapter Implementation Guide

Required Methods: Implement all methods defined in Section 4.3. Each method must be tested against the URIB Rail Adapter Test Suite.

Testing Requirements:

- Unit tests for `to_canonical()` and `from_canonical()` covering at least 50 distinct event types per rail.
- Integration tests connecting to a rail testnet and verifying end-to-end Rail Object production.
- Finality status tests: verify that `UNCONFIRMED → CONFIRMING → FINALIZED` transitions occur correctly.
- Taproot Root publication and verification tests against the URIB test Taproot tree fixtures.
- Error handling tests: verify all error codes from Section 11.7 are correctly raised.

10.3 Creating a New Canonical Object

```
// Pseudocode — Creating a new Canonical Object from a raw event  function
create_canonical_object(   rail_id:       string,   raw_event:     object,
semantic_tags: [string],   creator_thread: string ) → CanonicalObject:    //
Step 1: Normalize via Rail Adapter   rail_adapter = get_rail_adapter(rail_id)
rail_obj = rail_adapter.read_event(raw_event.tx_hash, raw_event.output_index)
// Step 2: Assign Semantic ID   concept_def =
resolve_concept_from_tags(semantic_tags, rail_obj.payload)   sid =
compute_sid(concept_def)  // urn:urib:sid:{namespace}:{SHA3-
256(concept_def)[:32]}    // Step 3: Assign Ordinal   ordinal =
ordinal_registry.next_ordinal(rail_id)    // Step 4: Compute Payload Hash
canonical_payload = canonicalize_payload(rail_obj.payload)   payload_hash =
SHA3-256(canonical_payload)    // Step 5: Construct Canonical Object
canonical_id = uuid_v5(URIB_NAMESPACE, sid + ordinal_to_string(ordinal))
canonical_obj = CanonicalObject {     schema_version: "1.0",
canonical_id:   canonical_id,     semantic_id:    sid,     ordinal:
ordinal,     payload_hash:   payload_hash.hex(),     rail_anchors:   [{
rail_id, block_height: rail_obj.block_height,                          tx_hash:
rail_obj.tx_hash, output_index: rail_obj.output_index }],     created_at:
utc_now(),     version:        1,     status:         "ACTIVE",     metadata:
{       created_by: creator_thread,       tags:       semantic_tags,
description: null     }   }    // Step 6: Compute C-Hash (do not include
c_hash in signed payload)   canonical_obj.c_hash =
compute_chash(canonical_obj)    // Step 7: Register in Canonical Registry
canonical_registry.register(canonical_obj)    // Step 8: Update Semantic
Graph   semantic_graph.add_node(sid, semantic_tags, canonical_id)    return
canonical_obj
```

10.4 Registering a Bridge Commitment

```
// Pseudocode — Registering a new Bridge Commitment  function
register_bridge_commitment(   canonical_id:   string,   rail_from:
string,   rail_to:        string,   address_from:   string,   address_to:
string,   initiator_sk:   Ed25519PrivateKey ) → BridgeCommitment:    // Step
1: Fetch canonical object and validate status   obj =
canonical_registry.get(canonical_id)   assert obj.status == "ACTIVE"   assert
commitment_registry.active_commitment_for(canonical_id) == null    // Step 2:
Compute target representation on rail_to   obj_dst = rail_map.apply(obj,
rail_from, rail_to)  // φ(O, R_src, R_dst)    // Step 3: Compute nullifier
nullifier = SHA3-256(canonical_id_bytes || ordinal_bytes || rail_from_bytes)
// Step 4: Assign ordinal for BRIDGE_INIT event   ordinal_from =
ordinal_registry.next_ordinal(rail_from)    // Step 5: Construct commitment
commitment = BridgeCommitment {     commitment_id:    uuid_v5(URIB_NS,
canonical_id + ordinal_from_str),     canonical_id:     canonical_id,
semantic_id:      obj.semantic_id,     rail_from:        rail_from,
rail_to:          rail_to,     address_from:     address_from,
address_to:       address_to,     state_hash_from:  SHA3-
256(canonical_serialize(obj)).hex(),     state_hash_to:    SHA3-
256(canonical_serialize(obj_dst)).hex(),     ordinal_from:     ordinal_from,
nullifier:        nullifier.hex(),     status:           "INITIATED",
finality_window_src: finality_windows[rail_from],     finality_window_dst:
finality_windows[rail_to],     dispute_deadline: utc_now() + 7 * 24 * 3600
// 7-day dispute window   }    // Step 6: Sign commitment   sig =
ed25519_sign(initiator_sk, canonical_serialize(commitment))
commitment.witness_sigs = [{ signer_id: initiator_thread_id,
public_key: initiator_pk.hex(),                                    signature:
sig.hex() }]    // Step 7: Publish nullifier to rail_from   nullifier_anchor
= rail_adapter(rail_from).publish_nullifier(nullifier)
commitment.nullifier_anchor = nullifier_anchor   commitment.status =
"PENDING"    // Step 8: Submit to Commitment Batcher
commitment_batcher.submit(commitment)    // Step 9: Register commitment
commitment_registry.register(commitment)    // Step 10: Update canonical
object status   obj.status = "BRIDGING"   canonical_registry.update(obj)
return commitment
```

10.5 Verifying a Cross-Rail Invariant Programmatically

```
// Example: Verify Invariant I-5 (Non-Duplication) for a canonical object
function verify_non_duplication(canonical_id: string) → boolean:   obj =
canonical_registry.get(canonical_id)   active_rails = []    for anchor in
obj.rail_anchors:     adapter = get_rail_adapter(anchor.rail_id)
nullifier = compute_nullifier(canonical_id, obj.ordinal, anchor.rail_id)
if not nullifier_registry.is_published(nullifier):       // Object is
potentially active on this rail       status =
adapter.get_object_status(canonical_id, anchor)       if status == "ACTIVE":
active_rails.append(anchor.rail_id)    if len(active_rails) > 1:
log_error("ERR_DUPLICATION_DETECTED", canonical_id, active_rails)     raise
DuplicationError(canonical_id, active_rails)    return len(active_rails) <= 1
```

10.6 Common Integration Errors

Error	Cause	Resolution
`ERR_CHASH_COLLISION`	C-Hash computed for a new object matches an existing registry entry	Verify that the new object is genuinely distinct. If it is a duplicate, do not register it. If distinct, investigate SHA3-256 implementation for bugs.
`ERR_ORDINAL_NON_MONOTONIC`	Ordinal assigned is ≤ current high-water mark for the rail	Re-request a new ordinal from the Ordinal Registry. Do not reuse ordinals.
`ERR_SEMANTIC_INCONSISTENCY`	SID on mapped object does not match source SID	Verify Rail Mapping function preserves SID. Check SID computation for both objects.
`ERR_THREAD_CONTINUITY_BROKEN`	Thread event parent_hash does not match predecessor	Fetch the latest thread event from the registry and recompute parent_hash from it.
`ERR_BRIDGE_NO_ACTIVE_OBJECT`	Attempted to bridge an object that is not in ACTIVE status	Check object status before initiating bridge. Wait for prior bridge to SETTLE or REVERT.
`ERR_FINALITY_WINDOW_NOT_ELAPSED`	Commitment finalized before finality window elapsed	Implement proper wait logic. Use conservative finality windows from Section 8.2.
`ERR_WITNESS_SIGNATURE_INVALID`	Ed25519 signature	Verify that the signer's public key is current (check for recent KEY_ROTATION events). Verify

Error	Cause	Resolution
	verification failed	signing payload is canonically serialized.
`ERR_SEMANTIC_LOSSLESSNESS_VIOLATED`	Mapped object is missing semantic tags present on source	Update Rail Mapping function to copy all source tags to destination. Re-run mapping.

10.7 Reference Implementation Notes — Base44 Platform

The URIB reference implementation exists as a fully working demo within the Base44 platform. All seven bridge stack layers have active primitive implementations. Developers integrating with the Base44 URIB deployment should note the following:

- The Base44 deployment exposes a REST API for all Canonical Registry, Ordinal Registry, and Commitment Registry operations. API documentation is maintained separately from this specification.
- The Ordinal Registry in the Base44 deployment uses a distributed ledger backend. Ordinal requests have a maximum latency of 200ms under normal load.
- The Commitment Batcher in the Base44 deployment operates on a 60-second batch window. High-priority commitments may request priority batching (additional fee applies).
- Rail Adapters for Bitcoin (btc-mainnet, btc-testnet), Ethereum (eth-mainnet, eth-sepolia), XRPL (xrpl-mainnet), and Solana (solana-mainnet) are pre-deployed. Custom rail adapters can be registered via the Rail Adapter Registry API.
- The Base44 Semantic Graph is pre-populated with standard concept definitions for common asset types (UTXO, ERC-20, SPL token, XRP trust line) and identity types (DID, OAuth2, X.509). Custom concepts can be registered via the Semantic Graph API.

11. Formal Reference Tables

> *This section provides all formal reference tables for implementors. These tables are normative and constitute part of the URIB specification.*

11.1 Canonical Object Field Reference

Field Name	Type	Required	Description	Validation Rules
`schema_version`	string	**REQUIRED**	Schema version of this canonical object	Must equal "1.0" for this specification version
`canonical_id`	string (UUID v5)	**REQUIRED**	Globally unique ID for this canonical object	Must be valid UUID v5; computed from SID + ordinal
`semantic_id`	string (URN)	**REQUIRED**	Semantic ID linking to Semantic Graph node	Must match pattern urn:urib:sid:{ns}:{32-hex-chars}
`ordinal`	OrdinalNamespace	**REQUIRED**	Global ordinal assignment for creation event	Must be unique in Ordinal Registry; all three fields required
`payload_hash`	hex string (64 chars)	**REQUIRED**	SHA3-256 of canonical payload bytes	Must be 64 lowercase hex chars; recomputable from payload

Field Name	Type	Required	Description	Validation Rules
`rail_anchors`	array[RailAnchor]	**REQUIRED**	Rail locations where this object is anchored	Minimum 1 entry; each entry must have valid rail_id, block_height, tx_hash
`created_at`	ISO8601 UTC string	**REQUIRED**	UTC creation timestamp	Must be valid ISO8601 with Z suffix; must not be in the future
`version`	integer	**REQUIRED**	Monotonic version counter	Must start at 1; incremented on every state transition
`status`	enum	**REQUIRED**	Current lifecycle status	Must be one of: ACTIVE, BRIDGING, NULLIFIED, TOMBSTONED
`c_hash`	hex string (64 chars)	**REQUIRED**	Canonical Hash (not included in signed payload)	Computed per Section 3.3.3; verified on all reads
`metadata.created_by`	string (thread URN)	**REQUIRED**	ThreadZero ID of the creating identity	Must resolve to a registered, ACTIVE ThreadZero
`metadata.tags`	array[string]	**REQUIRED**	Semantic tag(s) from taxonomy	Minimum 1 tag; all tags must be from approved taxonomy
`metadata.description`	string or null	OPTIONAL	Human-readable description	Max 500 characters; UTF-8 NFC normalized
`metadata.custom`	object or null	OPTIONAL	Extension fields	Must be hash-stable (lexicographically-sorted keys); no nesting deeper than 3 levels

11.2 Semantic Tag Taxonomy

Tag	Description	Valid Relationships	Example Use Cases
IDENTITY	A logical entity with an associated ThreadZero and active presence on at least one rail	OWNS, CONTROLS, IS_DELEGATED_BY, REVOKES, ASSERTS, VERIFIED_BY	User DID, organization identity, service account, IoT device identity
ASSET	A transferable, quantifiable resource with defined value semantics and trackable ownership state	OWNED_BY, TRANSFERRED_TO, LOCKED_IN, DERIVED_FROM, COLLATERALIZES	BTC UTXO, ERC-20 balance, SPL token, XRP, NFT, real-world asset token
CLAIM	A verifiable assertion made by one IDENTITY about another canonical object; may carry expiry	ASSERTS, REFUTES, VERIFIED_BY, EXPIRES_ON, SUPERSEDES	KYC attestation, verifiable credential, compliance certification, age proof
EVENT	A state change or occurrence at a specific ordinal; immutable once recorded	CAUSED_BY, RESULTS_IN, PRECEDES, FOLLOWS, CONTAINS	Token transfer, bridge initiation, key rotation, smart contract execution
RELATIONSHIP	A typed, directed association between two or more canonical objects	LINKS, MEDIATES, GOVERNS, DELEGATES_TO, REPORTS_TO	Ownership link, custody relationship, trust delegation, organizational hierarchy
POLICY	A set of machine-readable rules governing object behavior, access, or permissible state transitions	GOVERNS, ENFORCES, PERMITS, DENIES, OVERRIDES	Transfer policy, spending limit, AML rule set, access control list
COMMITMENT	A cryptographic commitment representing a pledge of future state, progress-tracked through bridge lifecycle	COMMITS_TO, PROVEN_BY, SETTLED_BY, REVERTS, BACKS	Bridge commitment, smart contract escrow, time-locked obligation

11.3 Rail Adapter Required Methods

Method Name	Signature	Description	Error Codes
`rail_id()`	() → string	Returns canonical rail identifier	None
`adapter_version()`	() → string	Returns semver adapter version	None
`read_event()`	(tx_hash, output_index?) → RailObject	Reads and normalizes a specific on-chain event	ERR_EVENT_NOT_FOUND, ERR_RAIL_UNAVAILABLE
`read_block()`	(block_height) → RailObject[]	Returns all URIB-relevant events in a block	ERR_BLOCK_NOT_FOUND, ERR_RAIL_UNAVAILABLE
`get_finality_status()`	(tx_hash, block_height) → FinalizationStatus	Returns current finalization status of a tx	ERR_TX_NOT_FOUND
`publish_taproot_root()`	(taproot_root, batch_id) → RailAnchor	Publishes a Taproot Root to the rail	ERR_PUBLICATION_FAILED, ERR_INSUFFICIENT_FUNDS
`publish_nullifier()`	(nullifier) → RailAnchor	Publishes a nullifier to the rail	ERR_NULLIFIER_DUPLICATE, ERR_PUBLICATION_FAILED
`to_canonical()`	(rail_object, semantic_id?) → CanonicalObject	Converts Rail Object to Canonical Object	ERR_CANONICALIZATION_FAILED, ERR_INVALID_RAIL_OBJECT
`from_canonical()`	(canonical_obj) → RailObject	Converts Canonical Object to	ERR_MAPPING_FAILED, ERR_UNSUPPORTED_SEMANTIC_TYPE

Method Name	Signature	Description	Error Codes
		rail-native format	
`verify_taproot_inclusion()`	(leaf_hash, merkle_path, taproot_root) → boolean	Verifies Merkle proof for commitment leaf	ERR_INVALID_PROOF

11.4 Bridge Commitment State Transitions

From State	To State	Trigger	Validator
—	INITIATED	Bridge request submitted by authorized identity	Identity Registry + ThreadZero auth check
INITIATED	PENDING	Nullifier confirmed on rail_from	rail_from Adapter: get_finality_status(nullifier_tx)
PENDING	ANCHORED	Taproot Root confirmed on both rails	Both Rail Adapters: get_finality_status(TR_tx)
ANCHORED	PROVEN	Merkle proof verified + witnesses signed	verify_bridge_commitment() algorithm (Section 6.4)
PROVEN	FINALIZED	Both rail finality windows elapsed	Automated finality monitor: time-based assertion
FINALIZED	SETTLED	Destination object activated; settlement proof generated	Settlement Proof Validator: verify SettlementProof schema
ANCHORED / PROVEN	DISPUTED	Valid Dispute Notice submitted before dispute_deadline	Dispute Registry: validate Dispute Notice signature and grounds
DISPUTED	REVERTED	Arbitration upholds dispute	Arbitration Panel or on-chain arbitration contract
DISPUTED	FINALIZED	Arbitration rejects dispute (commitment valid)	Arbitration Panel or on-chain arbitration contract
PENDING / ANCHORED	REVERTED	Commitment expires (beyond 3× max finality window)	Commitment Expiry Monitor: time-based assertion

11.5 Cross-Rail Ordinal Namespace

Field	Type	Uniqueness Scope	Description
`rail_id`	string	Global (URIB-assigned)	Canonical identifier for the rail on which this ordinal was assigned. Must be registered in the Rail Adapter Registry.
`epoch`	uint64	Global (all rails)	Monotonically-increasing epoch counter shared across all URIB rails. Epoch boundaries are synchronized via epoch anchor events published to all rails.
`sequence`	uint64	Per (rail_id, epoch)	Monotonically-increasing sequence number within the given epoch on the given rail. Resets to 1 at each new epoch. Never zero.

11.6 Finality Window Reference by Rail

Rail	Rail ID	Nominal Finality	URIB Conservative Window	Finality Trigger
Bitcoin Mainnet	btc-mainnet	~60 min	90 min	6 confirmed blocks
Bitcoin Testnet	btc-testnet	~60 min	90 min	6 confirmed blocks
Ethereum Mainnet	eth-mainnet	~12.8 min	15 min	2 finalized epochs
Ethereum Sepolia	eth-sepolia	~12.8 min	15 min	2 finalized epochs
XRPL Mainnet	xrpl-mainnet	~4 sec	30 sec	1 validated ledger round
Solana Mainnet	solana-mainnet	~13 sec	30 sec	32 confirmed slots
IPFS (any)	ipfs-*	N/A	Depends on anchor rail	Anchoring rail's finality applies
HTTPS (any)	https-*	N/A	Never used as settlement rail	N/A

11.7 Error Code Registry

Code	Name	Description	Remediation
`E1001`	ERR_CHASH_COLLISION	Computed C-Hash already exists in registry for a different object	Verify object uniqueness; debug canonicalization implementation
`E1002`	ERR_INVALID_CANONICAL_SCHEMA	Canonical Object does not conform to schema v1.0	Validate object against canonical schema before submission
`E1003`	ERR_PAYLOAD_HASH_MISMATCH	Recomputed payload_hash does not match stored value	Re-canonicalize payload; check for non-deterministic serialization
`E2001`	ERR_ORDINAL_NON_MONOTONIC	Ordinal sequence ≤ current high-water mark for rail	Request new ordinal from Ordinal Registry
`E2002`	ERR_EPOCH_SEQUENCE_VIOLATION	Ordinal epoch is less than current epoch	Verify epoch synchronization; request ordinal in current epoch
`E2003`	ERR_ORDINAL_NOT_FOUND	Ordinal Proof references an ordinal not in the registry	Verify ordinal was assigned before proof was generated
`E3001`	ERR_SEMANTIC_INCONSISTENCY	SID resolves to different concepts on different rails	Verify Semantic Graph node for this SID; check mapping function
`E3002`	ERR_SEMANTIC_LOSSLESSNESS_VIOLATED	Mapped object missing semantic tags from source	Update mapping function to copy all source tags

Code	Name	Description	Remediation
E3003	ERR_SID_NOT_FOUND	Semantic ID not registered in Semantic Graph	Register concept in Semantic Graph before creating objects with this SID
E4001	ERR_THREAD_CONTINUITY_BROKEN	Thread event parent_hash does not match predecessor hash	Fetch latest thread event; recompute parent_hash
E4002	ERR_THREAD_TOMBSTONED	Attempt to append event to a TOMBSTONED thread	No remediation; tombstoned threads are terminal
E4003	ERR_THREAD_MERGE_PROHIBITED	Event references parent hashes from two different threads	Thread merging is prohibited; use Semantic Graph RELATIONSHIP instead
E5001	ERR_TAPROOT_INCLUSION_FAILED	Merkle proof does not validate against Taproot Root	Recompute Merkle path; verify leaf hash computation
E5002	ERR_TAPROOT_ROOT_NOT_ANCHORED	Taproot Root not confirmed on required rail	Wait for rail confirmation or resubmit TR publication
E6001	ERR_BRIDGE_NO_ACTIVE_OBJECT	Source object is not in ACTIVE status	Check object status; wait for prior commitment to settle or revert
E6002	ERR_BRIDGE_DUPLICATE_COMMITMENT	Active commitment already exists for this canonical_id	Wait for existing commitment to complete before initiating new one
E6003	ERR_BRIDGE_FINALITY_WINDOW_NOT_ELAPSED	Commitment finalized before finality	Implement correct wait logic; use

Code	Name	Description	Remediation
		window elapsed	conservative window values
`E7001`	ERR_WITNESS_SIGNATURE_INVALID	Ed25519 signature verification failed	Verify signer's current public key; check signing payload canonicalization
`E7002`	ERR_WITNESS_UNAUTHORIZED	Signer's ThreadZero is not authorized for this operation	Use an authorized signer; verify signer's ThreadZero status
`E8001`	ERR_DUPLICATION_DETECTED	Canonical object found in ACTIVE state on more than one rail	Emergency: initiate reconciliation procedure; investigate nullifier publication
`E8002`	ERR_NULLIFIER_DUPLICATE	Nullifier already published for this canonical_id on this rail	Object already committed; check commitment status before resubmitting
`E9001`	ERR_RAIL_UNAVAILABLE	Rail Adapter cannot connect to underlying rail	Check rail node availability; implement retry with backoff
`E9002`	ERR_RAIL_NOT_REGISTERED	Rail ID not found in Rail Adapter Registry	Register Rail Adapter for this rail before use

12. Appendices

Appendix A — C-Hash Computation Reference Implementation

```
// URIB C-Hash Computation — Reference Implementation (Pseudocode) // Language: Implementation-agnostic pseudocode // Prerequisite: SHA3-256 implementation (FIPS 202 compliant)  function compute_chash(canonical_obj: CanonicalObject) → hex_string:    // Step 1: Encode canonical_id as UTF-8 bytes   cid_bytes = utf8_encode(canonical_obj.canonical_id)    // Step 2: Encode ordinal as big-endian bytes   // epoch:    8 bytes big-endian uint64   // sequence: 8 bytes big-endian uint64   ordinal_bytes = big_endian_uint64(canonical_obj.ordinal.epoch)                 || big_endian_uint64(canonical_obj.ordinal.sequence)   // Note: rail_id is NOT included in ordinal bytes to preserve portability    // Step 3: Decode payload_hash hex string to raw bytes   payload_hash_bytes = hex_decode(canonical_obj.payload_hash)   // Must be exactly 32 bytes (SHA3-256 output)   assert len(payload_hash_bytes) == 32    // Step 4: Compute sorted anchors hash   // Sort rail_anchors by tx_hash hex string (lexicographic, ascending)   sorted_anchors = lexicographic_sort_by(canonical_obj.rail_anchors, key=lambda a: a.tx_hash)   // Concatenate sorted tx_hash bytes   anchor_concat = ""   for anchor in sorted_anchors:     anchor_concat = anchor_concat || hex_decode(anchor.tx_hash)   // Hash the concatenation   sorted_anchors_hash = SHA3-256(anchor_concat)   // Result: 32 bytes    // Step 5: Concatenate all components   preimage = cid_bytes            || ordinal_bytes        // 16 bytes            || payload_hash_bytes   // 32 bytes            || sorted_anchors_hash  // 32 bytes   // Total preimage length: len(cid_bytes) + 80 bytes    // Step 6: Compute C-Hash   c_hash_bytes = SHA3-256(preimage)   // Step 7: Return as lowercase hex string (64 chars)   return hex_encode(c_hash_bytes).lower()   // Verification Helper: Verify C-Hash on read function verify_chash(canonical_obj: CanonicalObject) → boolean:   computed = compute_chash(canonical_obj)   return computed == canonical_obj.c_hash.lower()
```

Appendix B — Taproot Root Computation Reference Implementation

```
// URIB Taproot Root Computation — Reference Implementation (Pseudocode) // Prerequisite: SHA3-256 implementation  function compute_leaf_hash(commitment_leaf: CommitmentLeaf) → bytes[32]:   // Canonical serialization: lexicographically-sorted JSON, no whitespace, UTF-8 // IMPORTANT: Exclude the leaf_hash field itself from serialization leaf_copy = clone(commitment_leaf)   delete leaf_copy.leaf_hash   serialized = canonical_json_serialize(leaf_copy)   return SHA3-256(utf8_encode(serialized))   function compute_taproot_root(commitment_leaves: CommitmentLeaf[]) → hex_string:    // Step 1: Validate input   assert len(commitment_leaves) >= 1    // Step 2: Compute leaf hash for each leaf   leaf_hashes = [compute_leaf_hash(leaf) for leaf in commitment_leaves]    // Step 3: Sort leaf hashes lexicographically (ascending byte order)   sorted_hashes = sort(leaf_hashes, comparator=lexicographic_bytes_ascending)    // Step 4: Build Merkle tree bottom-up   current_level = sorted_hashes    while len(current_level) > 1:  next_level = []     i = 0     while i < len(current_level):       left = current_level[i]       // If odd number at this level, duplicate the last node       right = current_level[i+1] if i+1 < len(current_level) else current_level[i]       // Node hash: SHA3-256 of left || right (raw bytes, not hex)       node_hash = SHA3-256(left || right)  next_level.append(node_hash)       i += 2     current_level = next_level  // Step 5: Return Taproot Root as hex string   TR = current_level[0]   return hex_encode(TR).lower()   function compute_merkle_path(   all_leaf_hashes: bytes[][],  // Sorted list of all leaf hashes   target_hash:     bytes[]  // The leaf hash to prove ) → MerklePathEntry[]:   // Produces an ordered list of sibling hashes and positions   // for verifying target_hash's inclusion in the Taproot Root    path = []   current_level = sorted_copy(all_leaf_hashes)   current_hash = target_hash    while len(current_level) > 1:     idx = index_of(current_level, current_hash)  // Find sibling     if idx % 2 == 0:  // current is left child  sibling_idx = idx + 1 if idx + 1 < len(current_level) else idx       position = "RIGHT"     else:             // current is right child       sibling_idx = idx - 1       position = "LEFT"     path.append({ hash: hex_encode(current_level[sibling_idx]), position: position })     // Move up  next_level = []     i = 0     while i < len(current_level):       l = current_level[i]       r = current_level[i+1] if i+1 < len(current_level) else current_level[i]       next_level.append(SHA3-256(l || r))       i += 2  current_hash = SHA3-256(       current_level[idx] if idx % 2 == 0 else current_level[idx-1],       current_level[idx] if idx % 2 == 1 else current_level[idx+1] if idx+1 < len(current_level) else current_level[idx] )     // Simplified: move current_hash to its parent position  current_hash = next_level[idx // 2]     current_level = next_level    return path   function verify_merkle_proof(   leaf_hash:    hex_string,  merkle_path:  MerklePathEntry[],   taproot_root: hex_string ) → boolean:  current = hex_decode(leaf_hash)   for entry in merkle_path:     sibling = hex_decode(entry.hash)     if entry.position == "RIGHT":       current = SHA3-256(current || sibling)     else:  // "LEFT"       current = SHA3-256(sibling || current)   return hex_encode(current).lower() == taproot_root.lower()
```

Appendix C — ThreadZero Genesis Event Construction

```
// ThreadZero Genesis Event Construction — Reference Implementation
(Pseudocode) // Prerequisite: Ed25519 key generation, SHA3-256, UUID v5
function create_threadzero(   namespace:   string,    // Conceptual namespace
(e.g., "identity", "service")   metadata:    object,    // Arbitrary metadata
for the identity   ordinal_registry: OrdinalRegistry,   rail_adapter:
RailAdapter ) → { thread_id: string, genesis_event: ThreadEvent, keypair:
Ed25519KeyPair }:    // Step 1: Generate Ed25519 key pair   (sk, pk) =
ed25519_generate_keypair()    // Step 2: Construct genesis payload
genesis_payload = {     "public_key":  hex_encode(pk),     "namespace":
namespace,     "created_at":  utc_now_iso8601(),     "metadata":    metadata
}   // Canonicalize and hash   payload_bytes   =
utf8_encode(canonical_json_serialize(genesis_payload))   payload_hash    =
SHA3-256(payload_bytes)    // Step 3: Assign ordinal from Ordinal Registry
ordinal = ordinal_registry.next_ordinal(rail_adapter.rail_id())    // Step 4:
Construct genesis event (parent_hash = null hash)   NULL_HASH = "00" * 32  //
32 zero bytes as hex   genesis_event = ThreadEvent {     schema_version:
"1.0",     thread_id:       "",     // Placeholder; computed below
event_ordinal:   ordinal,     event_type:      "GENESIS",     parent_hash:
NULL_HASH,     payload_hash:    hex_encode(payload_hash),     rail_anchor:
null,   // Set after publication     witness_sigs:    [],     // Set after
signing     derived_from:    null   }    // Step 5: Compute ThreadZero ID
// Thread ID is derived from the genesis event bytes BEFORE signing
genesis_bytes_unsigned = utf8_encode(canonical_json_serialize(genesis_event))
thread_id = "urn:urib:thread:" + hex_encode(SHA3-256(genesis_bytes_unsigned))
genesis_event.thread_id = thread_id    // Step 6: Sign genesis event
genesis_bytes_to_sign = utf8_encode(canonical_json_serialize(genesis_event))
signature = ed25519_sign(sk, genesis_bytes_to_sign)
genesis_event.witness_sigs = [{     signer_id:  thread_id,    // Self-signed
at genesis     public_key: hex_encode(pk),     signature:
hex_encode(signature)   }]    // Step 7: Publish to rail; obtain rail anchor
rail_anchor = rail_adapter.publish_thread_genesis(genesis_event)
genesis_event.rail_anchor = rail_anchor    // Step 8: Verify publication
finality (poll until FINALIZED)   wait_for_finality(rail_adapter,
rail_anchor.tx_hash, rail_anchor.block_height)    // Step 9: Register in URIB
Identity Registry   identity_registry.register(thread_id, genesis_event,
rail_anchor)    // Step 10: Return   return {     thread_id:     thread_id,
genesis_event: genesis_event,     keypair:       { secret_key: sk,
public_key: pk }     // IMPORTANT: Store sk in HSM immediately; never log or
transmit sk   }
```

Appendix D — Semantic Graph Query Language (SGQL)

SGQL is a minimal formal query language for traversing and querying the URIB Semantic Graph. It is inspired by SPARQL and Cypher but is restricted to operations safe for content-addressed, append-only DAGs.

```
// SGQL Formal Grammar — EBNF Notation  sgql_query     = select_clause ,
from_clause , [ where_clause ] , [ limit_clause ]  select_clause  = "SELECT"
, select_list select_list    = ("*" | select_item) , { "," , select_item }
select_item    = variable | property_access property_access = variable , "."
, identifier  from_clause    = "FROM" , "GRAPH" , graph_ref graph_ref      =
"urib:semantic_graph" | sid_literal  where_clause   = "WHERE" , condition , {
("AND" | "OR") , condition } condition      = node_pattern | edge_pattern |
property_filter  node_pattern   = "NODE" , "(" , variable , [":" , tag_list]
, ")" tag_list       = tag_name , { "|" , tag_name } tag_name       =
"IDENTITY" | "ASSET" | "CLAIM" | "EVENT"                | "RELATIONSHIP" |
"POLICY" | "COMMITMENT"  edge_pattern   = "EDGE" , "(" , variable , ")" , "-
[" , rel_type , "]->"                , "(" , variable , ")" rel_type       =
identifier  // e.g., OWNS, ASSERTS, COMMITS_TO, etc.  property_filter =
variable , "." , identifier , comparator , literal_value comparator      =
"==" | "!=" | ">" | "<" data-id="1873" | ">=" | "<=" data-id="1874" |
"CONTAINS" literal_value   = string_literal | integer_literal | sid_literal |
boolean  limit_clause   = "LIMIT" , integer_literal  // Examples: // SELECT
n, n.semantic_id FROM GRAPH urib:semantic_graph //   WHERE NODE (n:IDENTITY)
AND n.status == "ACTIVE" //   LIMIT 100  // SELECT e FROM GRAPH
urib:semantic_graph //   WHERE NODE (a:IDENTITY) AND NODE (b:ASSET) AND EDGE
(a) -[OWNS]-> (b) //   AND a.semantic_id ==
"urn:urib:sid:identity:a3f7c291e8b04d12"  // SELECT c FROM GRAPH
urib:semantic_graph //   WHERE NODE (c:COMMITMENT) AND c.status ==
"FINALIZED"  // Reserved keywords: SELECT, FROM, GRAPH, WHERE, AND, OR, NODE,
//                    EDGE, LIMIT, CONTAINS // Variables: $varname (dollar-
prefixed identifiers) // SID literals: angle-bracket wrapped:
<urn:urib:sid:...>
```

Appendix E — Changelog & Version History

Version	Date	Author	Summary of Changes
1.0	2026-07-09	leon (Principal Architect)	Initial release — authoritative canonical specification. All seven UBS layers defined. Full glossary, invariants, bridge commitment lifecycle, identity continuity, settlement equivalence, security model, integration guide, reference tables, and appendices.
—	—	—	*(Future versions to be recorded here. Each version bump requires a signed commit by the Principal Architect and a full invariant re-validation pass.)*

Appendix F — Conformance Checklist

An implementation must satisfy ALL items in this checklist to claim URIB v1.0 conformance. Each item must be independently verified by a qualified auditor.

LAYER IMPLEMENTATION

- ☐ Layer 0: All participating rails have registered, audited Rail Adapter implementations.
- ☐ Layer 1: Rail Object schema v1.0 is produced correctly for all supported event types on all rails.
- ☐ Layer 2: Canonicalization algorithm produces identical output for identical inputs across all environments.
- ☐ Layer 2: C-Hash computation is exactly as specified in Section 3.3.3 and Appendix A.
- ☐ Layer 2: C-Hash uniqueness is enforced at registration time.
- ☐ Layer 3: All canonical objects carry a valid SID resolvable in the Semantic Graph.
- ☐ Layer 3: Semantic Graph is append-only and content-addressed.
- ☐ Layer 4: ThreadZero Genesis Event is constructed exactly as specified in Section 3.5.3 and Appendix C.
- ☐ Layer 4: All Thread Events carry valid parent_hash, ordinal, rail_anchor, and witness_sigs.

- ☐ Layer 4: Identity Continuity Proofs are computable and verifiable for any pair of thread events.
- ☐ Layer 4: Key rotation follows the dual-signature procedure specified in Section 7.6.
- ☐ Layer 5: Taproot Root computation is exactly as specified in Section 3.6.3 and Appendix B.
- ☐ Layer 5: Taproot Root is published to all participating rails for every commitment batch.
- ☐ Layer 5: Merkle proof verification (verify_merkle_proof) is implemented correctly.
- ☐ Layer 6: Ordinal Registry enforces strict monotonicity and global uniqueness.
- ☐ Layer 6: Epoch synchronization events are published to all rails at each epoch boundary.

INVARIANT COMPLIANCE

- ☐ I-1 (Canonical Uniqueness): C-Hash collision detection is active at registration.
- ☐ I-2 (Semantic Consistency): SID preservation is verified at every rail mapping.
- ☐ I-3 (Ordinal Monotonicity): Ordinal Registry enforces high-water mark per rail.
- ☐ I-4 (Identity Continuity): parent_hash chain is validated for every new thread event.
- ☐ I-5 (Non-Duplication): Nullifier publication is required before destination activation.
- ☐ I-6 (Bridge Soundness): All bridge commitment validity conditions are checked.
- ☐ I-7 (Settlement Equivalence): Settlement proofs are generated and verifiable for all SETTLED commitments.
- ☐ I-8 (Semantic Losslessness): Post-mapping tag set containment assertion is active.

SECURITY

- ☐ SHA3-256 (FIPS 202 compliant) is used for all hash operations.
- ☐ Ed25519 (RFC 8032 compliant) is used for all digital signatures.
- ☐ All signing keys are stored in HSMs or equivalent secure enclaves in production.
- ☐ Nullifier registry is permanent and tamper-evident.
- ☐ Replay attack prevention is implemented for all commitment and thread event submissions.
- ☐ Cryptographic agility migration plan is documented.

OPERATIONAL

- ☐ Conservative finality window values (Section 8.2) are used for all supported rails.
- ☐ Automated cross-rail invariant monitoring runs at each epoch boundary.
- ☐ All canonical and commitment state transitions are logged in append-only, tamper-evident logs.
- ☐ Settlement proof archives are retained for ≥7 years.
- ☐ Dispute registration and arbitration workflow is implemented.
- ☐ Error codes from Section 11.7 are implemented and correctly raised.
- ☐ The full Conformance Checklist has been reviewed and signed off by the Principal Architect.
- ☐ A qualified third-party security audit has been completed and its findings addressed.
